DESIGN AND IMPLEMENTATION OF AN EXAMINATION VENUE AND SEAT ALLOCATION SYSTEM FOR THE FACULTY OF COMPUTING, BAYERO UNIVERSITY KANO.

BY

HAUWA KULIYA MALIKI

(CST/22/SWE/00858)

A PROJECT SUBMITTED TO THE DEPARTMENT OF SOFTWARE ENGINEERING, FACULTY OF COMPUTING, BAYERO UNIVERSITY KANO, NIGERIA. IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE AWARD OF THE BACHELOR OF SCIENCE (B.Sc.) DEGREE IN SOFTWARE ENGINEERING.

NOVEMBER, 2026

DECLARATION

I, Hauwa Maliki Kuliya, hereby declare that this project entitled "Design and Implementation of an Examination Allocation System for the Faculty of Computing, Bayero University Kano" is my original work and has not been submitted, either in part or in full, to any institution for the award of any degree or qualification. All sources of information used in this project have been duly acknowledged and referenced.

Hauwa Maliki Kuliya

CST/22/SWE/00858

Signature: _____ **Date: _____**

CERTIFICATION

This is to certify that the project titled "Exam allocation system for faculty of computing, Bayero University kano" is carried out by HAUWA KULIYA MALIKI with registration number CST/22/SWE/00858, a student of software engineering department, faculty of computing, Bayero University kano. This project has been submitted in partial fulfilment of the requirement for the award of BACHELOR OF SCIENCE DEGREE in software engineering.

Project Supervisor

Mr. Jamilu Salisu

Signature: _____

Date: _____

Project Coordinator

Mr. Babangida Sani

Signature: _____

Date: _____

Head of Department

Dr. Saratu Ilu

Signature: _____

Date: _____

External Examiner

Signature: _____

Date: _____

DEDICATION

This project is dedicated to Almighty Allah (SWT) for His infinite mercy, guidance, strength, and blessings throughout my academic journey. It is also dedicated to my beloved parents and family for their unwavering love, prayers, encouragement, and sacrifices, as well as to my lecturers, project supervisor and everyone who contributed to my academic success.

ACKNOWLEDGEMENT

All praise and gratitude are due to Almighty Allah (SWT) for His mercy, guidance, protection, and strength throughout my academic journey and for granting me the ability to complete this project successfully.

I express my sincere appreciation to my project supervisor, Malam Jamilu Salisu, for his valuable guidance, encouragement, constructive criticism, and continuous support throughout the course of this research. My appreciation also goes to the Project Coordinator, Malam Babangida Sani, the Head of the Department, Dr. Saratu Ilu, and all the lecturers in the Department of Software Engineering, Bayero University Kano, for their knowledge, guidance, and support during my studies.

I am deeply grateful to my parents, family members, friends, and colleagues for their prayers, encouragement, understanding, and unwavering support throughout my academic programme. Finally, I appreciate everyone who contributed directly or indirectly to the successful completion of this project. May Almighty Allah (SWT) reward you all abundantly. Ameen.

TABLE OF CONTENTS

CHAPTER ONE INTRODUCTION 1
1.1 Background of the study 1
1.2 Problem Statement 1
1.3 Aim and Objectives 2
1.4 Significance of the Study 2
1.5 Scope of the Study 3
1.6 Summary 4
CHAPTER TWO LITERATURE REVIEW 5
2.1 Introduction 5
2.2 Related Works 5
2.3 Summary of Reviewed Work 7
2.4 Summary 10
CHAPTER THREE METHODOLOGY 11
3.1 Introduction 11
3.2 Project Workflow 11
3.3 System Development Model 12
3.4 Analysis of Existing and Proposed System 12
3.4.1 Description of Existing System 12
3.4.2 Requirement Elicitation 13
3.4.3 Requirements Definition 14
3.4.4 Requirement Analysis – Use Case Diagram 15
3.5 System Design 18
3.5.1 Description of Proposed System 18

3.5.2 Architecture Design 20
3.5.3 Database Design 22
3.6 Summary 23
CHAPTER FOUR IMPLEMENTATION AND TESTING 24
4.1 Introduction 24
4.2 Implementation 24
4.2.1 Implementation Tools 24
4.2.2 Algorithms of Major Functionality 25
4.2.3 Description of System Operation 29
4.3 Testing 38
4.3.1 Unit Testing 38
4.3.2 Integration Testing 39
4.3.3 System Testing 39
4.3.4 Usability Testing 41
4.4 Summary 42
CHAPTER FIVE SUMMARY, CONCLUSION, LIMITATION AND RECOMMENDATION 43
5.1 Summary 43
5.2 Conclusion 44
5.3 Limitation 44
5.4 Recommendation 45
REFERENCES 46
Appendix A: Interview Questions Used to Gather Requirements (Section 3.4.2) 48
A.1 Examination Officer interview 48
A.2 Student interview 49

Appendix B: Usability Questionnaire and Results (Section 4.3.4) 50

Appendix C: Code of the Major Functions (Section 4.2.2) 51

C.1 Algorithm 1: Allocate students for one examination (allocations/services.py) 51

C.2 Algorithm 2: Detect clashes between examinations (exams/clash_service.py) 52

C.3 Algorithm 3: Publish a timetable (exams/views.py) 53

C.4 Algorithm 4: Import the students registered for a course (students/views.py) 54

LIST OF TABLES

Table 2.2: Summary of Related Works 8

Table 3.1: Use Case - Log In 16

Table 3.2: Use Case - Download Official Timetable 16

Table 3.3: Use Case - Allocate Venues and Seats 17

Table 4.1: Implementation tools 24

Table 4.2: Result of the automated tests 39

Table 4.3: System test cases 39

Table 4.4: Usability test results 41

LIST OF FIGURES

Figure 3.1: Project Workflow 11

Figure 3.2: Use Case diagram 15

Figure 3.3: Activity diagram for Allocating and Publishing an Examination Timetable 19

Figure 3.4: Layered Architecture of the System 20

Figure 3.5: Class Diagram 21

Figure 3.6: Entity Relationship Diagram of the Database 22

Figure 4.1: Exam Officer login page 30

Figure 4.2: Superadmin login page 30

Figure 4.3: Officer dashboard (System Overview 31

Figure 4.4: Students directory 32

Figure 4.5: Courses catalog 33

Figure 4.6: Examination venues 33

Figure 4.7: Academic departments 34

Figure 4.8: Seating list of an examination 35

Figure 4.9: Printed seating chart 35

Figure 4.10: Student dashboard 36

Figure 4.11: Printable personal timetable 37

Figure 4.12: Official examination timetables 37

Figure 4.13: Invigilator dashboard 38

ABSTRACT

Examination allocation in the faculty of computing is often error-prone, and students sometimes cannot find their venue or seat. This project aims to develop a web-based Examination Venue and Seat Allocation System for the Faculty of Computing, Bayero University Kano, that allocates students to venues and seats, prevents examination clashes and gives students access to their examination information. Chapter One introduces the study, its problem, aim, objectives, significance and scope. Chapter Two reviews ten related studies on examination hall and seating allocation and identifies the gap this project fills. Chapter Three describes the iterative and incremental development model, the analysis of the existing system through an interview with a member of the examination committee, and the design of the proposed system, including its architecture and database. Chapter Four presents the implementation of the system with Django and SQLite and its testing. Chapter Five gives the conclusion and recommendations

CHAPTER ONE
INTRODUCTION

1.1 Background of the study

The number of students in universities is increasing with every session. Examinations are one of the most important parts of education because they assess students' performance and skills. As the number of students grows, conducting examinations has become more difficult. It is therefore essential for universities and other higher institutions to find an effective way to organize all examination activities in line with examination guidelines and directives (Mora et al., 2023).

In recent years, examination activities such as allocating students to examination venues, arranging seats and assigning invigilators have often been done manually. These processes can be time-consuming and may lead to errors. Examples are incorrect student allocation, scheduling conflicts and difficulties in allocation. These challenges become more serious as the number of students increases (Vakil et al., 2025).

The technological advancements have made it possible to automate many processes within educational institutions. An examination allocation system will go a very long way in improving the overall examination processes and mitigate human errors and mistakes by automatically assigning students to various exam venues, allocating seat numbers randomly and assigning invigilators to various venues based on a given set of rules (Inamdar et al., 2018)

This work focuses on developing an Examination Allocation System that allocates students to examination venues and assigns invigilators to the respective venues. The system will randomly arrange students and assign each student a seat number. It will also provide students and invigilators with convenient access to relevant examination information through a dedicated web-based portal.

1.2 Problem Statement

Examination allocation in higher institutions is often slow and prone to errors such as incorrect student allocation, venue overcrowding, scheduling conflicts and uneven invigilator assignment (Anjum Assistant Professor et al., n.d.) . In the Faculty of Computing, Bayero University Kano, an allocation software is used, however, it has two main limitations. First, a student who checks

the allocation once cannot check again until after some hours, so students cannot confirm their venue and seats on demand. Secondly, some students cannot find their seats and venues until they meet the Examination officer in person. These problems were reported by students during interviews conducted for this study. The proposed system addresses them in two ways. Each student has a personal login and can view and download their examination timetable, venue and seat number at any time, with no waiting period. Every student registered for a course is given a venue and a numbered seat, and a timetable cannot be published if any student has no seat or any examination has no venue, so no student needs to meet the Examination Officer to find their seat. The system is intended to improve accuracy, and enhance the overall efficiency of examination management.

1.3 Aim and Objectives

To design and develop a web-based Examination Allocation System for the Faculty of Computing, Bayero University Kano.

The Objectives are:

i. To analyze the existing examination allocation system for the Faculty of Computing, Bayero University, Kano.

ii. To design and develop a web-based Examination Venue and Seat Allocation System for the Faculty of Computing Bayero University, Kano.

iii. To test the developed system to determine its effectiveness in improving the efficiency and accuracy of examination allocations.

1.4 Significance of the Study

This study will benefit the Faculty of Computing, Bayero University Kano by improving the efficiency of examination allocation through the automation of student, venue, and invigilator assignments. It will reduce human errors, save time, and improve the accuracy of examination management. The study will also serve as a useful reference for future researchers interested in examination management systems and related information systems.

1.5 Scope of the Study

The study focuses on designing and developing a web-based examination allocation system for supporting planning and management of examinations within the Faculty of Computing, Bayero University, Kano.

Features:

User authentication and role-based access: users log in with a username and password. The system has three roles: Examination Officer (administrator), student and invigilator. Each role sees only its own pages after login. The Examination Officer manages the system, students view and download their timetable and seat details, and invigilators view their assigned examinations. The Examination Officer can disable an account, and a disabled account cannot log in.

Timetable creation: the Examination Officer creates a timetable, which is a named set of examinations for one examination period (for example, "First Semester 2025/2026"). A new timetable starts as a draft.

Timetable upload: the Examination Officer uploads the examinations of the timetable (course, date and time) as a CSV or Excel file. The system does not generate examination schedules.

Student registration lists: the Examination Officer uploads the list of students registered for each course.

Venue allocation: the system assigns each examination to an examination venue with enough capacity for the students registered for the course.

Numbered seat allocation: the system gives each student a seat number in the assigned venue, and seat numbers are assigned randomly.

Invigilator assignment: the system assigns an invigilator to each examination venue.

Clash detection: the system checks for clashes within a timetable, and against published timetables, before publication.

Timetable publication: the Examination Officer publishes a draft timetable only when every student has a seat and every examination has a venue. Students can then log in to view and

download their examination timetable and seat details. A published timetable can be withdrawn (unpublished) or archived.

Users: Admin, Examination Officers, Invigilators and Students.

The project will not include mobile app development or integration with external systems. It will focus only on creating a web-based system for allocating and managing examinations in the faculty of computing.

1.6 Summary

This chapter introduced the background of the study, statement of the problem, aim and objectives, significance, and scope of the research. It highlighted the challenges associated with the current examination allocation errors, venue-related challenges, and inefficient invigilator assignment and the need for an automated solution. It outlined the intended users, major system features, and boundaries of the proposed system and also established the need for an automated Examination Allocation System to improve the efficiency and accuracy of examination management within the Faculty of Computing, Bayero University Kano.

CHAPTER TWO
LITERATURE REVIEW

2.1 Introduction

This chapter reviews ten earlier studies on examination hall allocation and seating arrangement systems. This chapter reviews ten earlier studies on examination hall allocation and seating arrangement systems. It groups them into web and application-based systems, and methods that use search algorithms such as genetic algorithms. It then compares them in a table and states the gap that this project fills.

2.2 Related Works

The works below are arranged in ascending order of year of publication.

Shelar et al.(2018) presented an Android-based Exam Cell Automation System. Students register their details, the administrator generates hall tickets and login credentials, and students view and print their tickets and update their details. The administrator generates the seating arrangement. The system centers on registration and hall tickets, and the paper gives no description of the allocation logic.

Dener and Calp (2018) proposed a two-stage genetic algorithm for central examinations. The first stage assigns courses to sessions of at most 150 minutes. The second assigns students to classrooms and halls so as to minimize unused seats, the number of supervisors needed and the number of buildings used. It was tested on a single scenario of three departments and 27 course entries, with no comparison against other methods, so its general usefulness is not shown.

Papaioannou (2018) presented oPESA, an online platform in which lecturers enter course, date and room details and add student lists or seating constraints. Students register with their student ID and receive a seat from an algorithm the authors describe as unbiased, ensuring randomness and fairness. Results can be viewed online or downloaded as spreadsheets. This is the closest to the present project in its use of uploaded student lists and random seating. Only the published abstract was available for this review.

Adetona et al. (2020) developed a PHP web application that stops students offering the same course from sitting together, a practice they call "giraffing". They modelled the problem as graph coloring

and k-partition and solved it with the Harmony Search Algorithm. The administrator logs in, enters class, course and student data, and prints the generated arrangement. It works for malpractice prevention but supports only an administrator role.

kashyap et al. (2021) used a genetic multi-optimization algorithm to place examinees in seats and rooms so as to reduce the chance of malpractice. They considered subject similarity, distance between examinees and field of vision, and treated the seating problem as NP-complete. They note that earlier methods rely on simple geometric arrangements and hard constraints. The approach suits large examination bodies but is more complex than a faculty-level system needs.

Anjum et al. (2021) developed a Windows-based application that generates seating arrangements from department and registration numbers and also assigns invigilators to halls. Student, hall and timetable details are stored, and a seating report is produced. The paper also surveys earlier systems, including a cloud-based one that allocates students according to hall capacity, and describes a genetic algorithm for seating.

Tung et al. (2021) combined compromise programming with a genetic algorithm to produce examination schedules for more than 2,500 students. Their multi-objective approach weighs cost, examination integrity and student convenience, and takes room layouts, examination durations and institutional rules into account. It addresses timetabling rather than the seating of individual students. Only the published abstract was available for this review.

Subhashini et al. (2022) built a PHP and MySQL web application that lets an administrator add students, staff, departments and subjects, schedule examination halls, allocate students to halls and assign staff to halls as invigilators. Students log in with an email and password to see their hall and seat number, and the administrator can reassign an invigilator when a staff member is absent. The authors describe the existing manual process as slow and error-prone, but the paper does not say how seats are chosen within a hall or how clashes between examinations are handled.

Adetona & Akintoye (2023) built a web application using Laravel, Vue.js and the Termii messaging service. It keeps students enrolled in the same course apart using a greedy graph coloring algorithm and a connection matrix, shows the arrangement graphically, and sends each student a text message five minutes before the examination with its time and location. Only the published abstract was available for this review.

Kibona (2026) proposed a mixed-course seating model that uses graph coloring and combinatorial design. Students from different courses share a room but are spatially separated. Using data for 5,175 students, the authors report needing 12 large rooms instead of 35 and halving invigilation needs from at least 70 to 36. The model targets universities with very limited space. Only the published abstract was available for this review.

2.3 Summary of Reviewed Work

This project does not claim a unique combination of features. It addresses the practical needs of the Faculty of Computing by combining uploaded course lists, random seating, clash detection and role-based access for students, invigilators and the Examination Officer in one system.

Table 2.2: Summary of Related Works

Author(s)	Year	Title	Research Focus	Programming Languages & Software Type (Methodology)	Key Functionalities Identified
Shelar et al.	2018	Exam cell automation system	Automating exam cell tasks and hall tickets	Android application (language not stated)	Student registration, hall tickets, administrator-generated seating arrangement
Dener and Calp	2018	Solving the exam scheduling problems in central exams with genetic algorithms	Exam scheduling and room assignment	Two-stage genetic algorithm (language not stated)	Assigns courses to sessions; assigns students to rooms to reduce unused seats, supervisors and buildings
Papaioannou et al.	2018	oPESA: Online platform for automatic exam-hall seat allocation	Automatic seat allocation	Web platform, random seat algorithm (language not stated)	Lecturers enter course and room details and student lists; students get random seats; results viewed online or exported as spreadsheets
Adetona et al.	2020	The development of a web-based application of examination seating arrangement for student	Preventing same-course students sitting together	PHP web application; Harmony Search Algorithm (graph colouring, k-partition)	Administrator login, entry of class, course and student data, printable seating arrangement
Kashyap et al.	2021	Exam seating allocation to prevent malpractice using genetic multi-optimization algorithm	Seating to reduce examination malpractice	Genetic multi-optimisation algorithm (language not stated)	Seating based on subject similarity, distance between examinees and field of vision
Anjum et al.	2021	Automation of exam hall allotment and seating arrangement	Hall allotment, seating and invigilators	Windows application (language not stated)	Seating from department and registration numbers, invigilator assignment, seating report
Tung et al. (2021) et al.	2021	Genetic algorithm for solving multi-objective optimization in examination timetabling problem	Examination timetabling	Compromise programming with a genetic algorithm	Schedules for 2,500+ students weighing cost, integrity and student convenience

Author(s)	Year	Title	Research Focus	Programming Languages & Software Type (Methodology)	Key Functionalities Identified
Subhashini et al.	2022	Online examination hall seating arrangement	Hall allocation and invigilator assignment	PHP and MySQL web application	Administrator adds students, staff and subjects; allocates halls and seats; reassigns invigilators; students view hall and seat
Adetona & Akintoye	2023	Design and implementation of an examination seating arrangement application to curb examination malpractice	Keeping same-course students apart	Laravel, Vue.js, Termii; greedy graph colouring	Graphical seating chart; SMS to each student five minutes before the examination
Kibona et al.	2026	Examination seating optimization using graph coloring and combinatorial design	Seating that saves rooms and invigilators	Graph colouring and combinatorial design (software not stated)	Mixed-course rooms; 12 large rooms instead of 35 and invigilators cut from at least 70 to 36

2.4 Summary

This chapter reviewed ten studies on examination hall allocation, seating arrangement and examination scheduling. Some are web or application-based systems that record students, halls and invigilators and generate seating, while others use algorithms such as genetic algorithms and graph coloring. Most support only an administrator, and few use uploaded student lists or give separate access to each type of user. This project fills these gaps for the Faculty of Computing, Bayero University Kano. The next chapter describes the methodology.

CHAPTER THREE
METHODOLOGY

3.1 Introduction

This chapter describes the methodology used to develop the Examination Venue and Seat Allocation System. It presents the project workflow and the system development model, analyses the existing and proposed systems, and describes the design of the proposed system, including its architecture and database.

3.2 Project Workflow

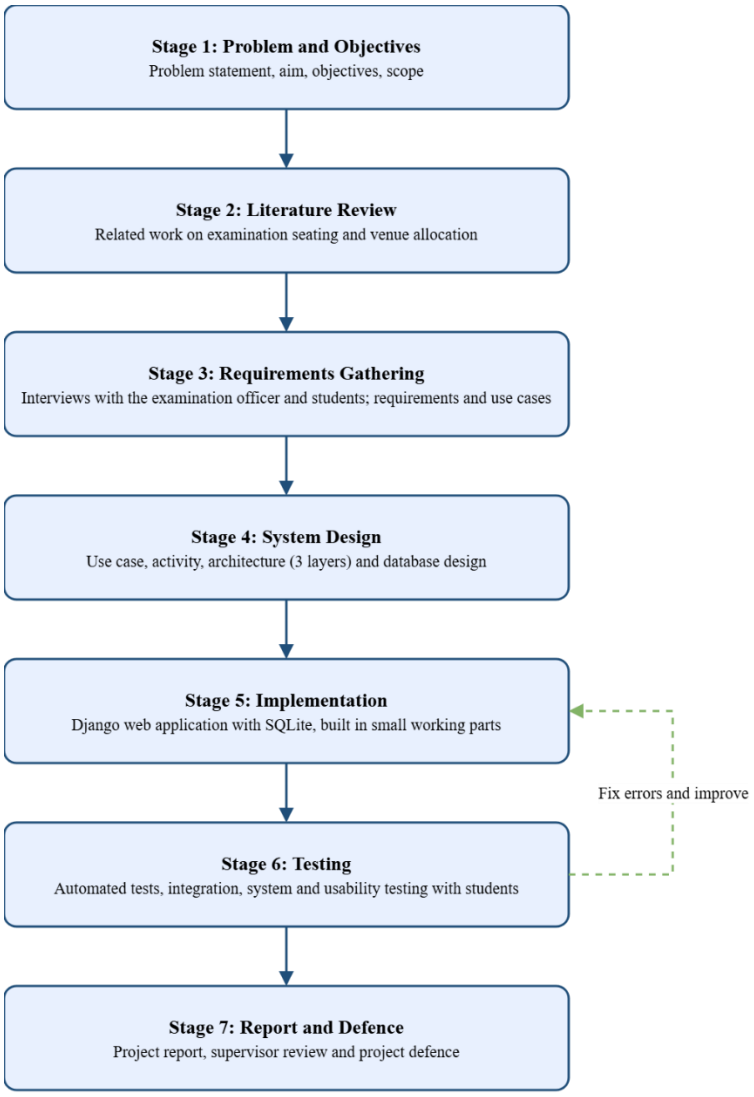


Figure 3.1: Project Workflow

The project follows the stages above, from the introduction to the defense. Each stage depends on the one before it.

3.3 System Development Model

The system was developed using an iterative and incremental model. The system was built in small working parts, such as students, courses, venues, exams and allocations. Each part was tested and improved before the next one was added.

This model was chosen because the requirements changed as the work progressed. For example, per-student course registration was replaced by uploading a list of registered students for each course, and seat allocation was changed from serial to random assignment. Spreadsheet imports and a timetable upload were also added after feedback. An iterative model allows such changes without restarting the project, and it lets the supervisor and the examination officer review each increment.

3.4 Analysis of Existing and Proposed System

3.4.1 Description of Existing System

The Faculty of Computing has about 2,000 students, 4 departments and about 30 courses per semester. Student and course records come from the school registration portal. Each department submits its list of courses, and the examination committee plans and approves the timetable. Venues are chosen by capacity. Seat numbers are given randomly, and invigilators are also chosen randomly, except that the lecturer of a course must be one of the invigilators for that course's examination. The Faculty already uses allocation software for this work. It saves time, and the examination committee runs it. People take part as follows:

- Examination committee (examination officer): plans and approves the timetable, chooses venues by capacity, runs the allocation software, and arranges a new venue if one fails.
- Students: check the list of students assigned to each venue to find their venue and where to sit for their examinations.
- Departments: each department submits its list of courses to the examination committee.

3.4.2 Requirement Elicitation

Requirements were gathered through a structured interview with the examination officer of the Faculty of Computing, Bayero University Kano. The interviewee has been a member of the Faculty's examination committee for one year. The interview took place on 26 October 2026 in Mallam Ubale's office, Faculty of Computing, Bayero University Kano. The interview questions are in Appendix A.1.

Findings:

- Key needs: the system should allocate venues and seats correctly and make a new arrangement if a venue or seat fails. It should assign invigilators automatically. Staff and students should log in to see their upcoming examinations or invigilation, with course, venue, time and duration, and to get the timetable. Students should see their venue and seat one hour before the examination. The officer should be able to supply data in Excel files and upload the timetable for students to download. The attendance sheet in use is acceptable.
- Challenges with the current process: some students cannot find their names on the venue list, and they often complain that they cannot find their venue or seat. This is mostly caused by carryover and spillover students. Venue failure is rare. Some invigilators also complain that their invigilation times are not convenient for them. This problem, and including the course lecturer among the invigilators of their own course, are not yet handled by the system and are left as future work.
- Interview with students: Students were also asked about the existing way of finding their examination venue and seat. The questions are given in Appendix A.2, and five students replied. They said they use the Faculty's existing software to find their venue and seat, and that after checking once they could not check again until some hours later, which made it difficult to confirm their venue. A student who went to the examination officer to find the seat said it took about 10 minutes. When asked what a new system should do, a student asked to be able to confirm the venue and seat on demand. These replies agree with the officer's report that some students cannot find their venue or seat.

3.4.3 Requirements Definition

User requirements

- The administrator shall be able to create and manage examination officer accounts including enabling and disabling.
- The examination officer shall be able to add, update and delete departments, courses, venues and invigilators.
- The examination officer shall be able to import students and course lists from Excel files.
- The examination officer shall be able to create the examination timetable and upload the official timetable as an image or PDF.
- The examination officer shall be able to allocate students to venues and seats for an examination, and to remove an allocation. The examination officer shall also be able to group examinations into a timetable timetable, publish it only when it has no clashes, and unpublish, archive or delete it.
- The invigilator shall be able to log in and view their examination duties. The system shall assign invigilators automatically.
- The student shall be able to log in, view their examinations and download the official timetable as a JPG or PNG image and also download their personal timetable in PDF format.
- The student shall be able to see the venue and seat number one hour before each examination. The release time shall be a configurable setting.
- Exam officers and administrators shall be able to change their own passwords

Non-functional requirements

- Role-based access: each user shall see only the pages that belong to their role.
- Security: passwords shall be stored hashed, and forms that change data shall be protected against forged requests.
- Correctness: the system shall not place two students on the same seat and the system shall detect students who are registered for two examinations that overlap in time on the same

day, and shall not allow a timetable with such clashes to be published. During allocation, the system shall not assign a venue or an invigilator to an examination if that venue or invigilator is already assigned to another examination that overlaps in time. Examinations that follow each other without overlapping may share a venue or an invigilator.

- Fairness: seat numbers shall be assigned in random order.
- Usability: the system shall work in a standard web browser, with simple pages for each role.
- Data integrity: imports shall report invalid rows without stopping the whole upload.

3.4.4 Requirement Analysis – Use Case Diagram

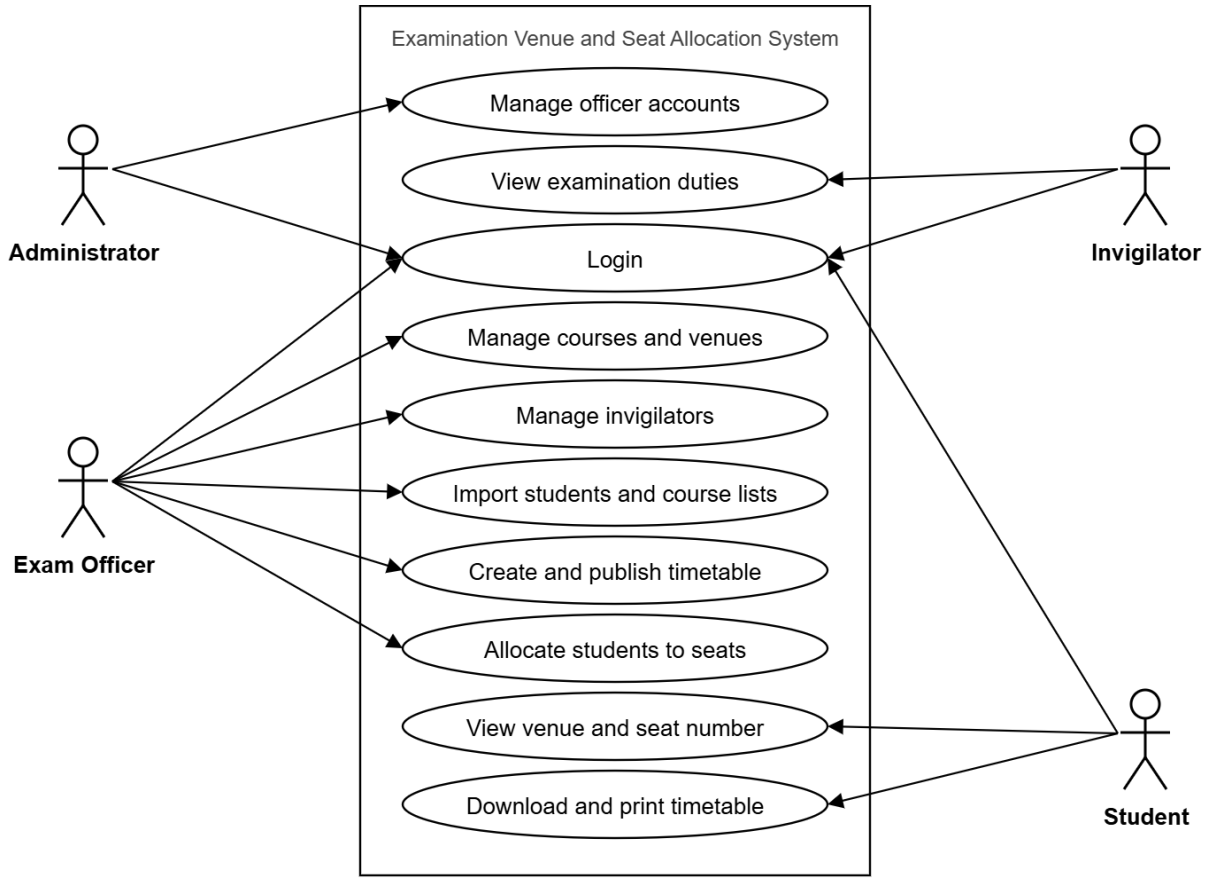


Figure 3.2: Use Case diagram

Every user logs in before using any of these functions.

Use Case Descriptions

Use case description for login

Table 3.1: Use Case - Log In

Use Case Name: Login **Priority: High**
Actor: Administrator, Exam Officer, Invigilator, Student
Description: This use case documents how a user logs into the examination venue and seat allocation system.
Trigger: External
Preconditions: The system is running and the user has an account.
Normal Course: 1) The system asks the actor to enter a username and password. 2) The actor enters the username and password. 3) The system validates the username and password (alternative course). 4) The system opens the pages that belong to the actor's role. 5) The use case ends.
Alternative Course: 1) The username or password is invalid. 2) The system asks the actor to enter them again. 3) The actor enters a valid username and password. 4) The use case continues at step 4.
Post Conditions: The user is logged in and sees only the pages for their role.
Exceptions: 1) The system displays "Please enter the correct username or password. Note that both fields may be case-sensitive".

Requirement Analysis- Use Case Description for timetable download

Table 3.2: Use Case - Download Official Timetable

Use Case Name: Download Official Timetable **Priority: Medium**
Actor: Student
Description: This use case documents how a student gets the official timetable and prints a personal timetable.
Trigger: External
Preconditions: The student is logged in and the timetable is uploaded by exam officer
Normal Course:

Use Case Name: Download Official Timetable **Priority: Medium**
1) The student opens the Official Timetables page. 2) The system lists the published timetables. 3) The student chooses a format (PNG or JPG). 4) The system gives the student the file. 6) The use case ends.
Post Conditions: The student has the timetable file
Exceptions: 1) If no timetable has been uploaded, the system shows an empty list.

Use case description for students seat allocation

Table 3.3: Use Case - Allocate Venues and Seats

Use Case Name: Allocate Students to Seats **Priority: High**
Actor: Exam Officer
Description: This use case documents how the system places the students of a course into venues and gives each a seat number.
Trigger: External
Preconditions: The officer is logged in. The examination, the course list and the venues exist, and the examination is in a draft timetable.
Normal Course: 1) The officer chooses an examination and selects Allocate. 2) The system finds the students on the course list. 3) The system places them in venues, starting with the smallest venue that can seat the whole group, and using the largest venue first only if no single venue is big enough, until all are placed. 4) The system gives each student a random seat number. 5) The system saves the allocation. 6) The use case ends.
Post Conditions: Every registered student has a venue and a seat number. No seat is used twice and no venue is over capacity.
Exceptions: 1) If the venues cannot hold all the students, the unseated students are left out and the timetable cannot be published until they are seated.

3.5 System Design

3.5.1 Description of Proposed System

The proposed system is a web application built with Django and an SQLite database. The administrator manages the examination officer accounts, including enabling and disabling them. The system shall create a login account for each student and each invigilator when the examination officer adds or imports their record. The username is the registration number or staff ID, and the account is linked to that student or invigilator record, so that each user sees only their own examinations or duties. Student and invigilator accounts are created automatically when their records are added or imported, with the registration number or staff ID as the username and initial password. The examination officer imports students, and for each course imports the list of students registered for it, from Excel files. The officer then sets up the examination timetable and venues.

When the officer runs the allocation for an examination, the system finds the students on that course's list and places them in venues, starting with the smallest venue that can seat the whole group. If no venue is large enough, it fills the largest venues one after the other until every student has a place. It numbers the seats in random order and checks that no student has two examinations that overlap in time, using each examination's start time and duration. Examinations are grouped into timetable timetables. A timetable can be edited only while it is a draft, and it can be published only when every registered student is seated and no student has a clash. Students see only published timetables, where they find their examinations, venue and seat number and can download the official timetable as an image. Invigilators log in to see their duties.

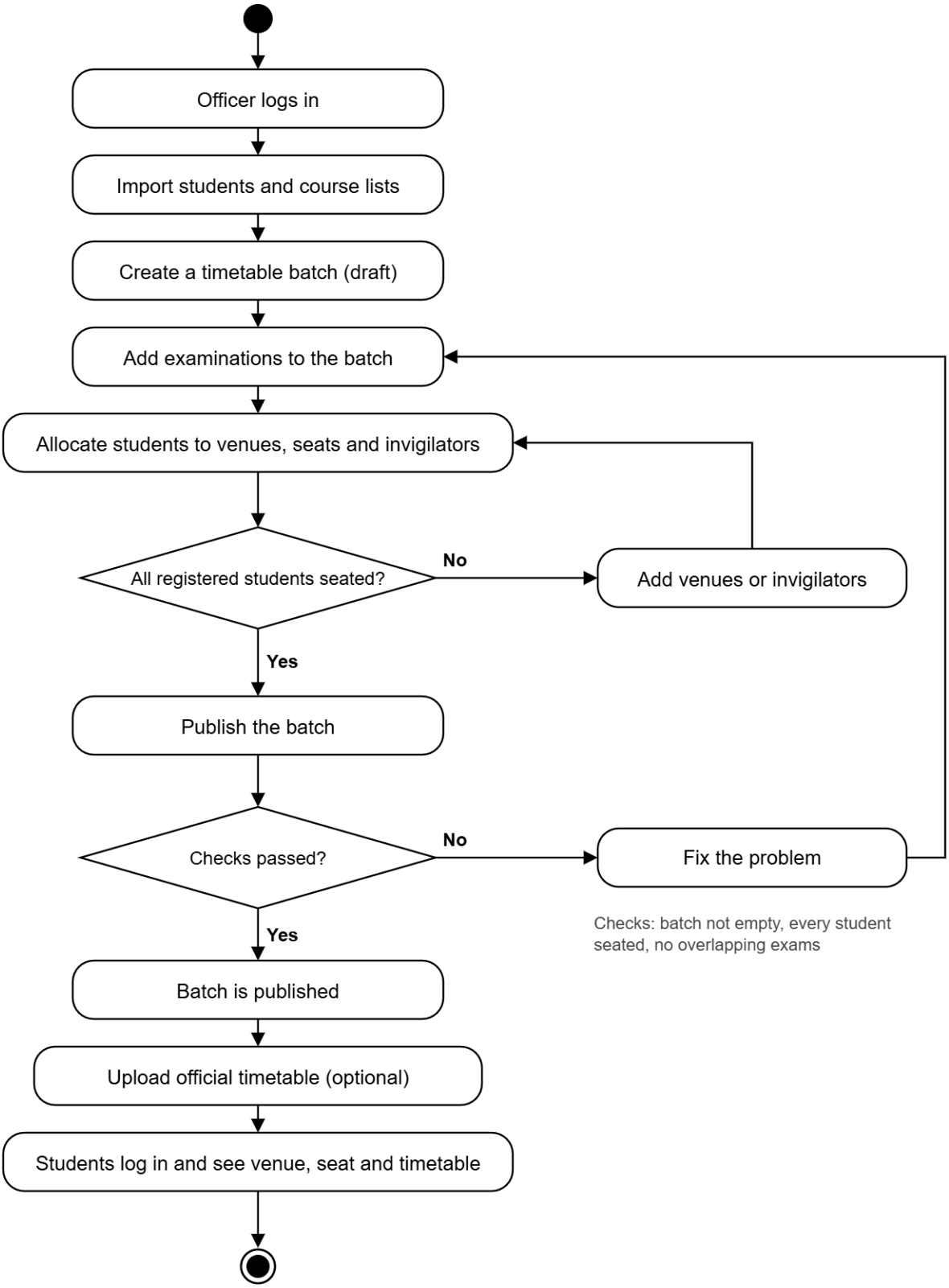


Figure 3.3: Activity diagram for Allocating and Publishing an Examination Timetable

3.5.2 Architecture Design

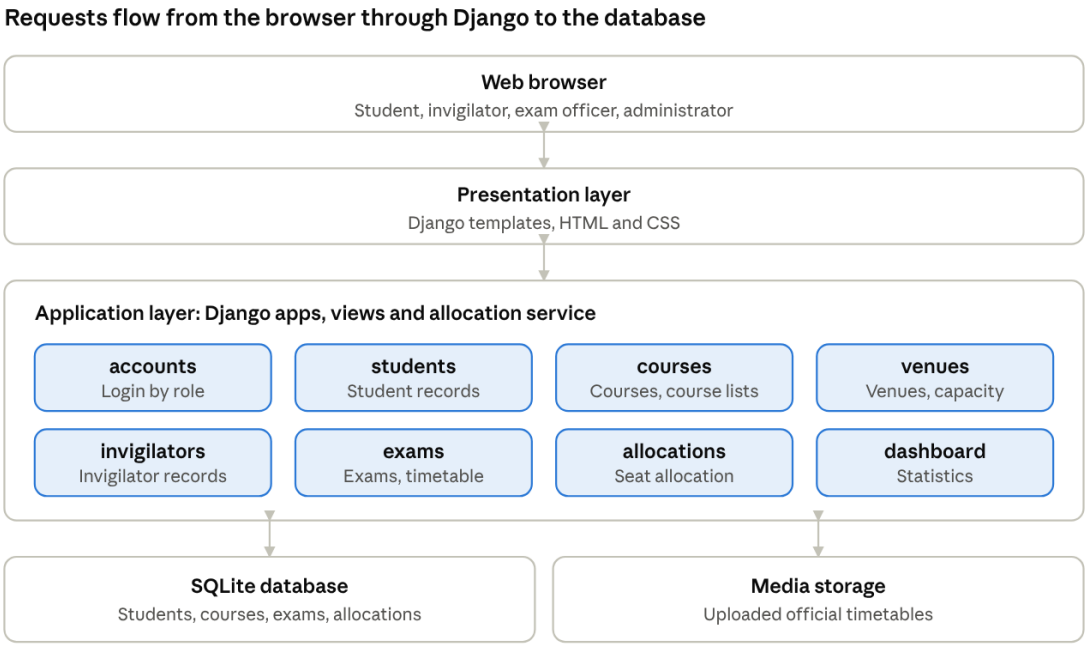


Figure 3.4: Layered Architecture of the System

The system has three layers. Users reach it through a web browser. Django templates present the pages, and the Django apps handle each part of the work, with the allocation logic kept in the allocation service. Data is stored in an SQLite database, and uploaded timetable files are kept in media storage.

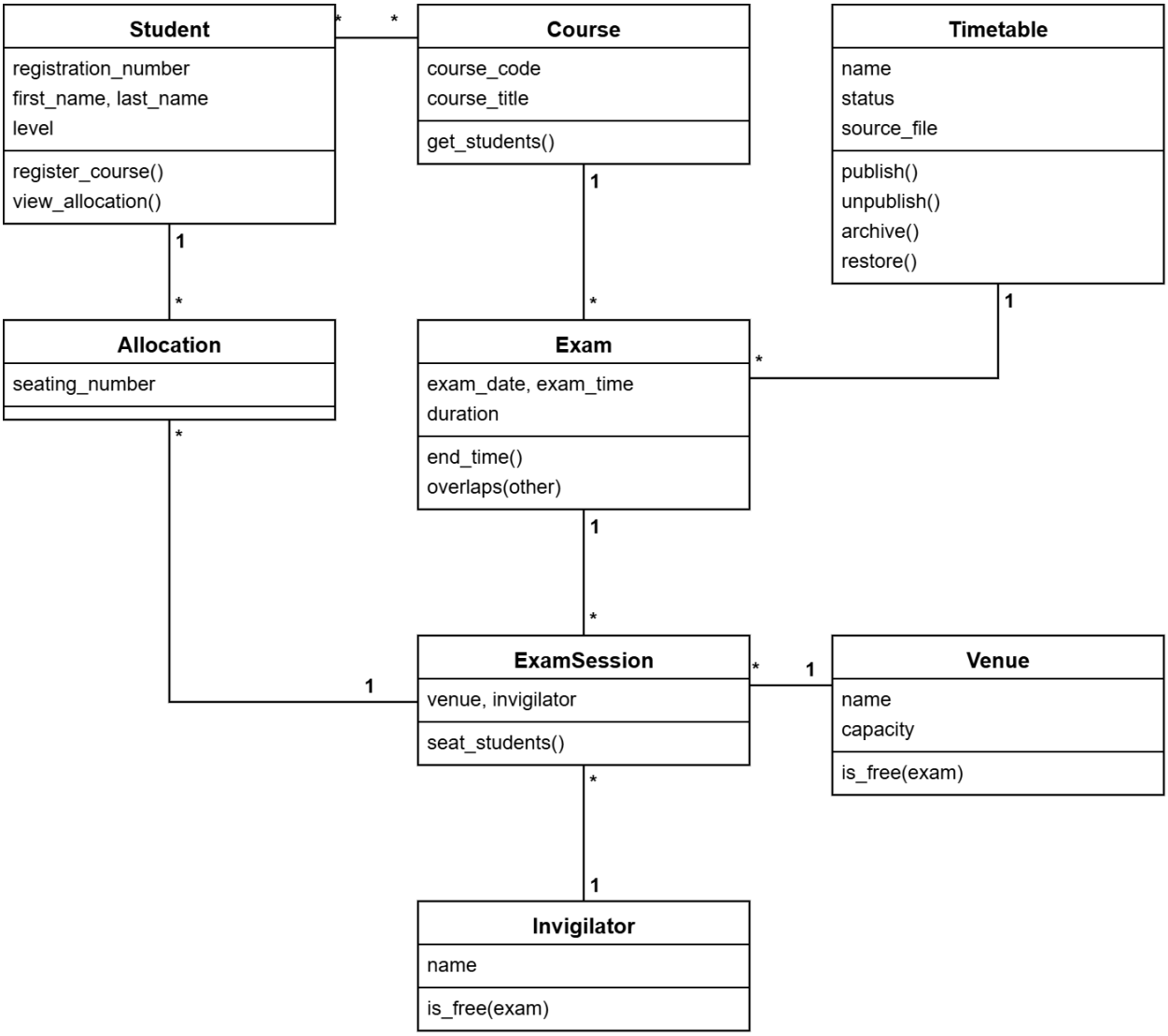
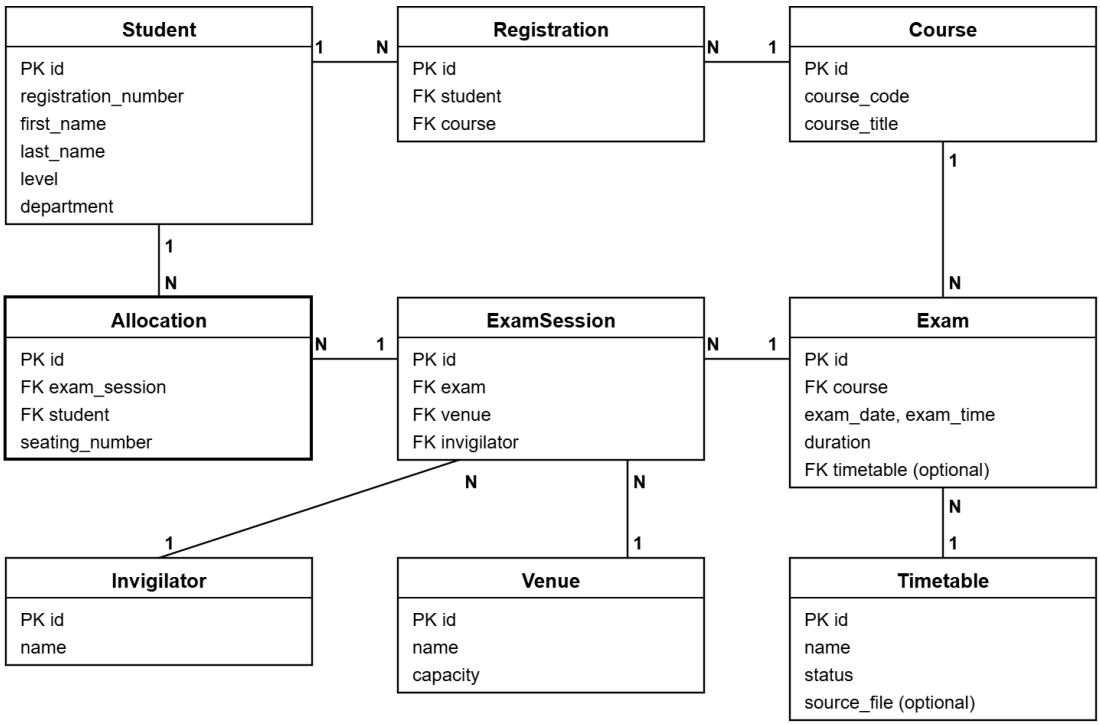


Figure 3.5: Class Diagram

3.5.3 Database Design



PK = primary key, FK = foreign key. 1 = one, N = many. Each line joins the "1" table to the "N" table that refers to it.

Figure 3.6: Entity Relationship Diagram of the Database

The diagram shows the main tables used for allocation. Each line joins a table to the one it refers to.

A student appears in many registrations, and a course appears in many registrations. Together they form the list of students registered for each course.

A course has many exams, and an exam has many exam sessions, one for each venue used.

An exam session joins an exam to one venue and one invigilator. A venue or an invigilator can be used by many exam sessions.

An exam may belong to one timetable, and a timetable can have an uploaded timetable file, which is stored as a field of the timetable. The status of a timetable is draft, published or archived.

An allocation joins one student to one exam session and gives a seating number.

The system also stores user accounts for each role. They are left out of the diagram for clarity.

3.6 Summary

This chapter described the project workflow and the iterative and incremental model used to build the system. It set out the requirements, analyzed them with a use case diagram, and described the proposed system, its three-layer architecture and its main database tables. The next chapter describes the implementation and testing of the system.

CHAPTER FOUR
IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter describes how the Examination Venue and Seat Allocation System was implemented and tested. It lists the tools used to build the system, explains the algorithms behind its major functions, and shows the working system with sample screens. It then presents the unit, integration, system and usability tests carried out to check that the system works correctly.

4.2 Implementation

This section lists the tools used to build the system, explains the algorithms behind its major functions, and shows the system in use with sample screens.

4.2.1 Implementation Tools

The tools below were used to build the system. Each was chosen because it is free, widely used and suited to a web application of this size.

Table 4.1: Implementation tools

Tool	Version	Used for	Why it was chosen
Windows	Windows	Operating system for development	The development computer runs Windows.
Visual Studio Code	Latest	Code editor	Free, light and has good support for Python, Django, HTML, CSS and JavaScript.
Python	3.12.7	Programming language for the back end	Easy to read and write, and has libraries for Excel, image and PDF files.
Django	6.0.3	Web framework	Provides user login, an admin area, forms and database tools, which speeds up development.
SQLite	Built in	Database	Needs no separate server, so it is simple to set up and move for a single-faculty system.
HTML and CSS	Standard	Page structure and styling	Standard languages for web pages that work in any browser.

Tool	Version	Used for	Why it was chosen
JavaScript	Standard	Page behaviour	Used for features such as the searchable course list on the exam form.
openpyxl	3.1.5	Reading Excel files	Lets the officer import students and course lists from Excel.
Pillow	Latest	Handling images	Creates the PNG and JPG files that students download from the timetable pages.
pypdfium2	Latest	Reading PDF files	Turns pages of an uploaded PDF timetable into images that students can download.
Git and GitHub	Latest	Version control	Keeps a history of changes and a backup of the code online.

4.2.2 Algorithms of Major Functionality

This section gives the algorithms of the major functions of the system. The first is the allocation of students to venues, seats and invigilators. The others are clash detection, publishing a timetable batch, and importing students and courses from Excel.

Algorithm 1: Allocate students for one examination

Input: an examination, the venues and the invigilators. Output: the exam sessions and the seat of each student.

1. Find all students registered for the course of the examination. If there are none, stop and report an error.

2. Shuffle the list of students into random order.

3. Delete any earlier exam sessions and seat allocations for this examination, so that running the allocation again is safe.

4. Order the venues. Venues that can seat the whole group come first, with the smallest suitable venue first and the least-used venue first when two have the same size. Venues that are too small come after them, largest first.

5. Remove every venue that is already booked for another examination whose time overlaps this one. An examination occupies its start time plus its duration.

6. Order the invigilators with the least-used first.

7. For each venue in order, while students remain:

    a. Pick the first invigilator who has not been used in this run and is not busy with an overlapping examination. If there is none, skip this venue.

    b. Create an exam session for the examination, the venue and the invigilator.

    c. Take as many students as the venue's capacity from the shuffled list.

    d. Give these students seat numbers 1, 2, 3 and so on, and save them.

8. If any students are left without a seat, cancel the whole allocation and report how many could not be seated.

9. Otherwise, save the allocation and finish.

Because the list of students is shuffled before the seat numbers are given, the seats are assigned randomly. The whole allocation is saved as one unit, so either everyone is seated or nothing is changed.

Algorithm 2: Detect clashes between examinations

Input: the set of examinations to compare. Output: the list of clashes, each with the student and the two examinations that overlap.

1. If there are fewer than two examinations, return an empty list, because nothing can clash.

2. Group the examinations by course.

3. Read the course registrations of these courses and note, for each student, the set of courses the student is registered for.

4. Keep only the students who are registered for more than one course.

5. Group the students who have exactly the same set of courses, so that each pair of courses is compared once for the whole group.

6. For each pair of courses in a group, compare each examination of the first course with each examination of the second course.

7. Two examinations overlap when they are on the same day and each starts before the other ends. An examination occupies its start time plus its duration. If one ends exactly when the other starts, they do not overlap.

8. For each overlapping pair, put the examination that starts first as the first of the pair, and add one clash to the list for every student in the group.

9. Return the list of clashes.

Only examinations that students can see are compared: the batch being checked and all published batches. A draft in another batch cannot clash with anything yet, and an archived batch is not used. A single examination is compared with the rest of its batch and the published batches. When a batch is published, only the pairs in which at least one examination belongs to that batch are reported. The overlap test is one shared function (exams_overlap), so the allocation and the clash check use the same rule. The registrations are read in one query and all comparisons are done in memory, so the check stays fast when there are many students.

Algorithm 3: Publish a timetable batch

Input: a timetable batch chosen by the officer. Output: the batch is published, or messages that say why it cannot be.

1. Accept the request only if it was sent as a form submission (POST). Otherwise, go back to the batch page.

2. If the status of the batch is not Draft, stop and report that only a draft can be published.

3. Get the examinations of the batch. If there are none, stop and report that the batch is empty.

4. For each examination, check that all its registered students can be seated. Collect every examination that cannot. If there is any, stop and list them, so that the officer can allocate seating and try again.

5. Run Algorithm 2 for the batch. If it returns any clash, stop and list each one with the student's registration number, the two courses, their times and the date.

6. If all the checks pass, change the status of the batch to Published and show a success message.

7. Students can now see the examinations of this batch.

A batch has one of three statuses: Draft, Published and Archived. Unpublish takes a published batch back to Draft. Archive hides a batch from students. Restore brings an archived batch back to Draft. Delete removes the batch with its examinations and seats, and the officer must confirm it first because it cannot be undone. The checks list every problem found, not only the first one, so the officer can fix them together.

Algorithm 4: Import the students registered for a course

Input: a course chosen by the officer, a CSV or Excel file with the columns reg_no and full_name, and the choice to add or replace. Output: the students registered for the course, and a report of the rows that were not accepted.

1. Read the file. If it cannot be read or has no header row, stop and report an error.

2. If the officer chose replace and seats have already been allocated for this course, stop and report that the list cannot be replaced.

3. Load all the students of the system into a lookup by registration number, in small letters, so that capital and small letters do not matter.

4. Find the students already registered for the course.

5. For each row of the file:

    a. If the registration number is empty, count the row as a bad row and go to the next row.

    b. If no student has this registration number, add the row to the list of unknown students.

    c. If the student was already listed earlier in the file, ignore the repeat.

    d. If the name in the file does not match the student's name in the system, add the row to the list of name mismatches. The order of the words and capital letters are ignored.

    e. If the student is already registered for the course, count the student as already registered. Otherwise, keep the student to be added.

6. If the officer chose replace and no row matched a student, stop and leave the existing list unchanged.

7. If the officer chose replace, keep the students to add and to remove, and show a preview. Nothing is saved until the officer confirms. The preview expires after one hour, and the officer may cancel it.

8. If the officer chose to add, register the new students in one step and show how many were added and how many were already registered.

9. When the officer confirms a replace, check again that no seats have been allocated for the course. If there are none, add the new students and remove the students who are not in the file, in one step, and show how many were added and removed.

In replace mode, nothing is deleted straight from the upload. The officer first sees exactly who will be added and who will be removed, and the change is made only after the officer confirms. A file in which no student matches the system is rejected, so a wrong registration number format can never empty a course list. The student list is imported in a similar way: each row is read, a student who already exists is skipped, a new student is saved, and a login is created for the student with the registration number as the username.

4.2.3 Description of System Operation

This section shows the main pages of the system with a short explanation of each. A user must log in first, and the pages shown depend on the role of the user: Superadmin, Exam Officer, Invigilator or Student. The steps of allocation, clash checking and publishing are explained in the algorithms in Section 4.2.2.

Exam Officer login page. The Exam Officer enters a username and a password and clicks the sign in button. The page has a link to the Superadmin login for users who have a Superadmin account.

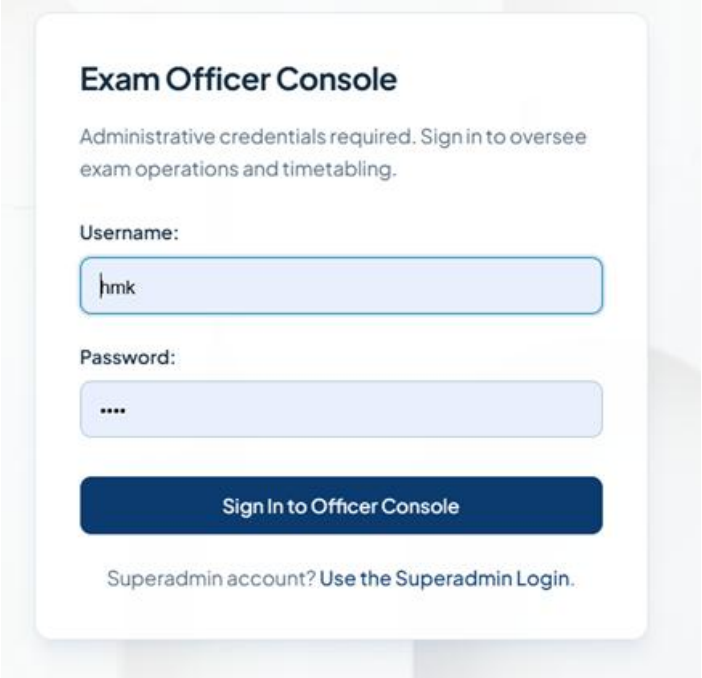


Figure 4.1: Exam Officer login page

Superadmin login page. This page is only for Superadmin accounts. It tells the Exam Officers to use the Exam Officer login page instead.

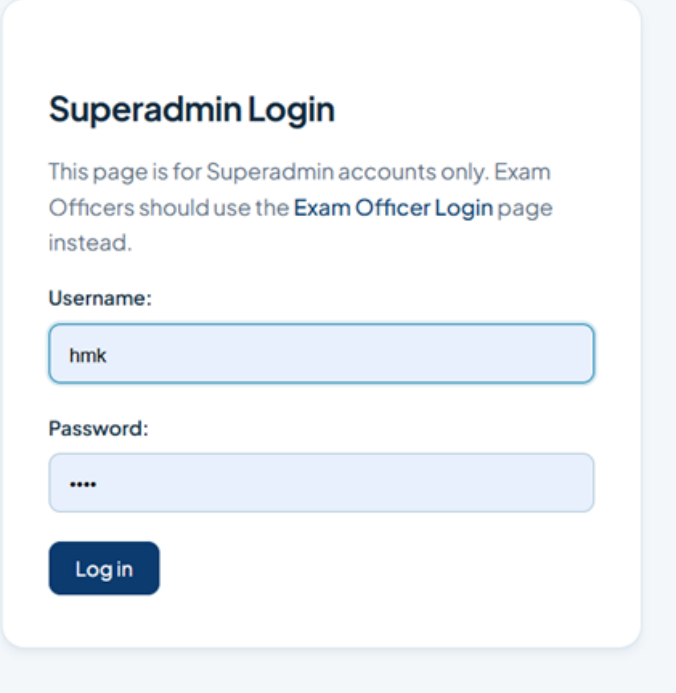


Figure 4.2: Superadmin login page

Officer dashboard. After login, the officer sees the System Overview. It shows the number of students, courses, venues, examinations, invigilators and allocations, and the number of students in each department. The menu on the left leads to the Students, Courses, Venues, Invigilators, Exams, Timetable, Allocations and Import Hub pages.

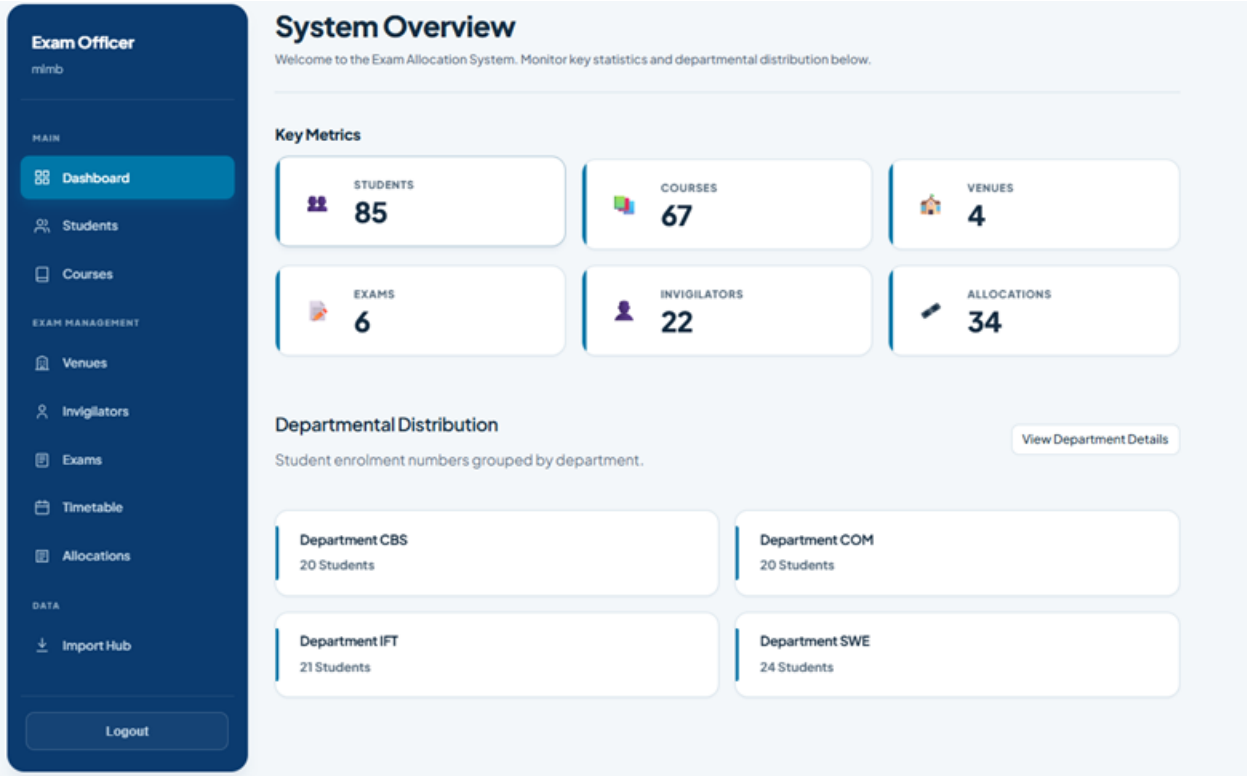


Figure 4.3: Officer dashboard (System Overview

Students directory. The officer can search for a student by registration number, name or email, and filter by department, level and gender. Each student has buttons to see the profile, register courses, edit and delete. The officer can add one student or import many students from a CSV or Excel file.

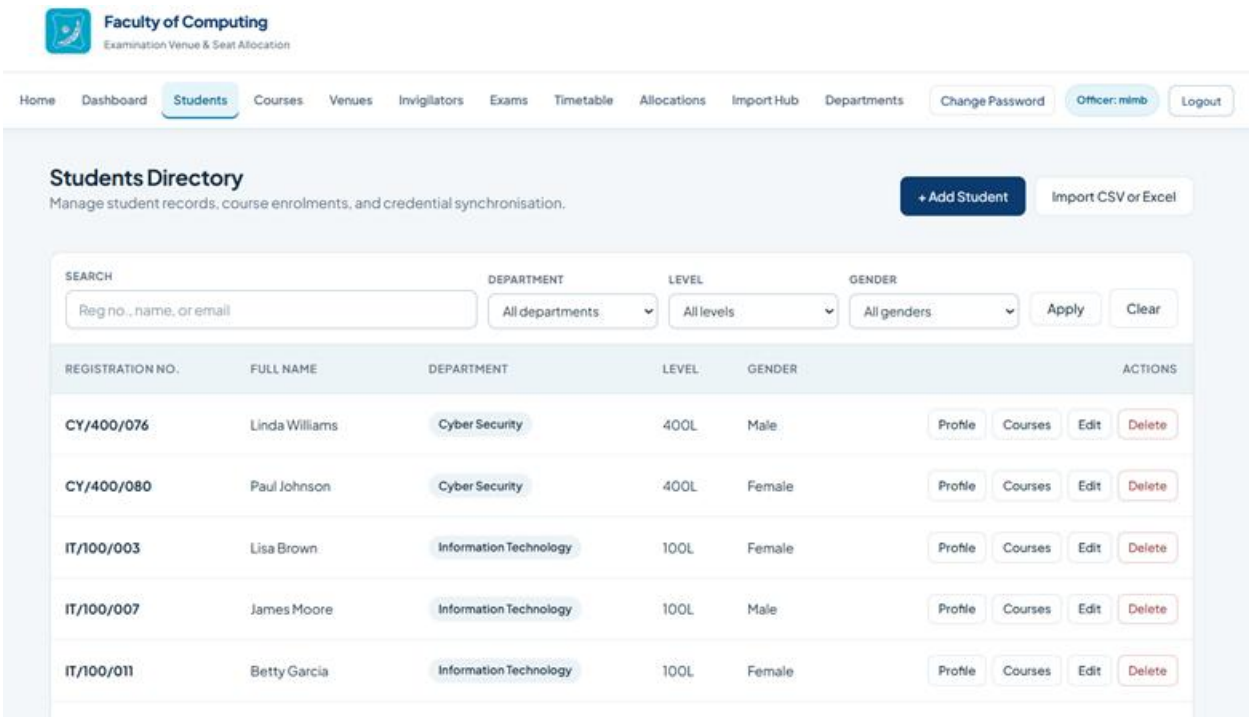


Figure 4.4: Students directory

Courses catalog. The officer can search the courses by code or title, and filter by department, level and semester. Each course can be edited or deleted. The officer can add a course or import courses from a CSV or Excel file.

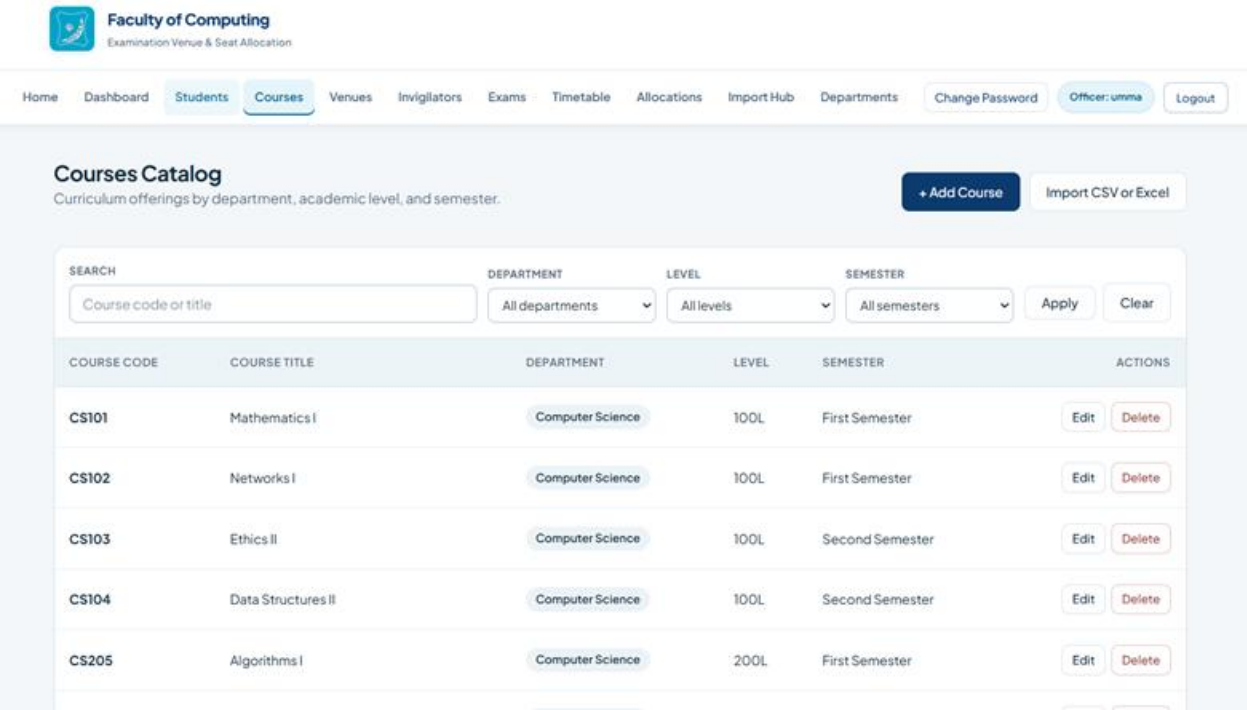


Figure 4.5: Courses catalog

Examination venues. This page lists the halls and classrooms with their seat capacity. The officer can search by name, filter by the smallest and largest capacity, and add, edit or delete a venue.

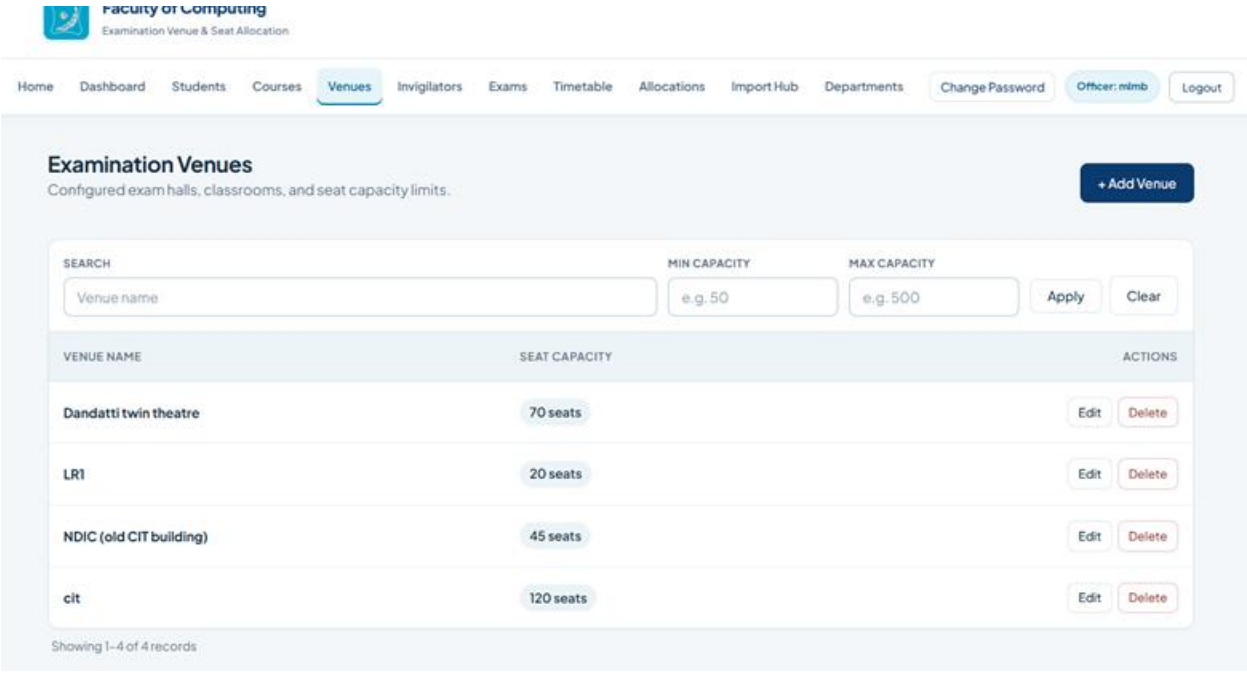


Figure 4.6: Examination venues

Departments. This page shows each department of the Faculty of Computing with its code, its full name, the number of students and the number of courses. The officer can open a department to see more.

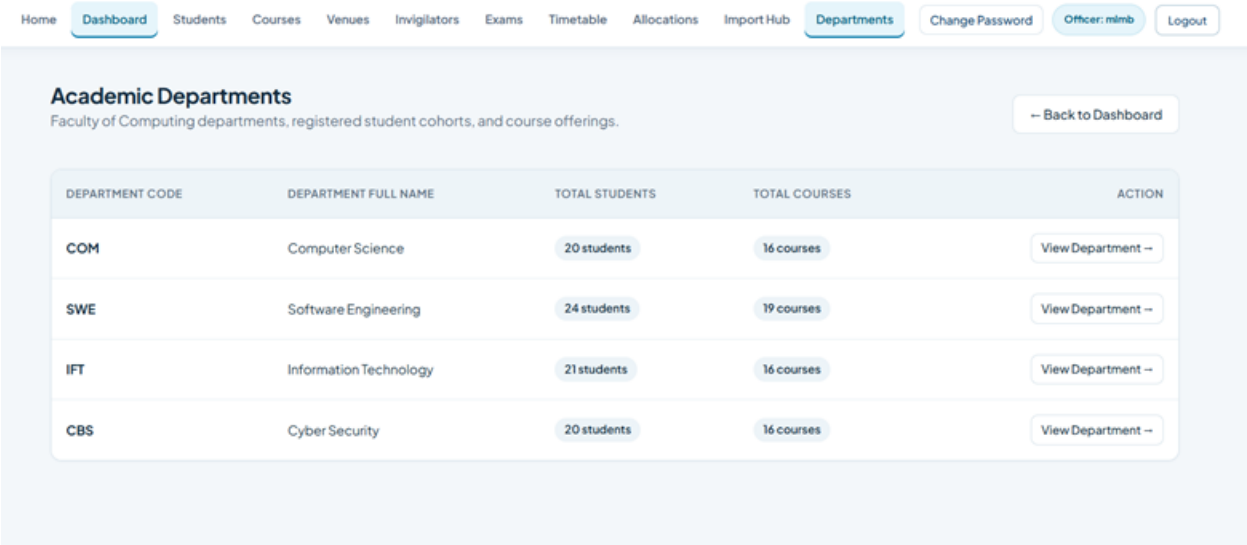


Figure 4.7: Academic departments

Seating list of an examination. After allocation, the officer can open an examination and see its venue, date, time and supervisor. The page lists each student with the seat number, registration number, name, department and level. The officer can search for a student, remove a student, or print the seating chart.

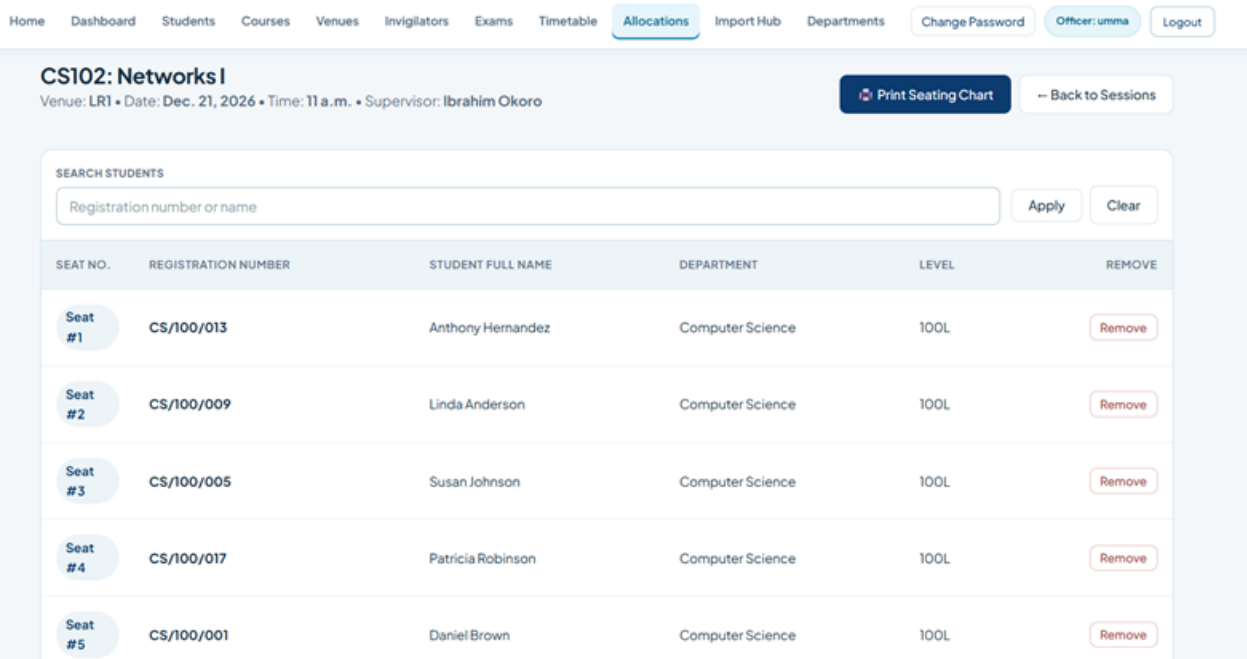


Figure 4.8: Seating list of an examination

Printed seating chart. The seating chart is made for the invigilator. It shows the course, the venue, the date, the time, the invigilator and the total number of students. It lists the seat, registration number and name of each student, with a column for the student's signature, and a line for the invigilator to sign and date.

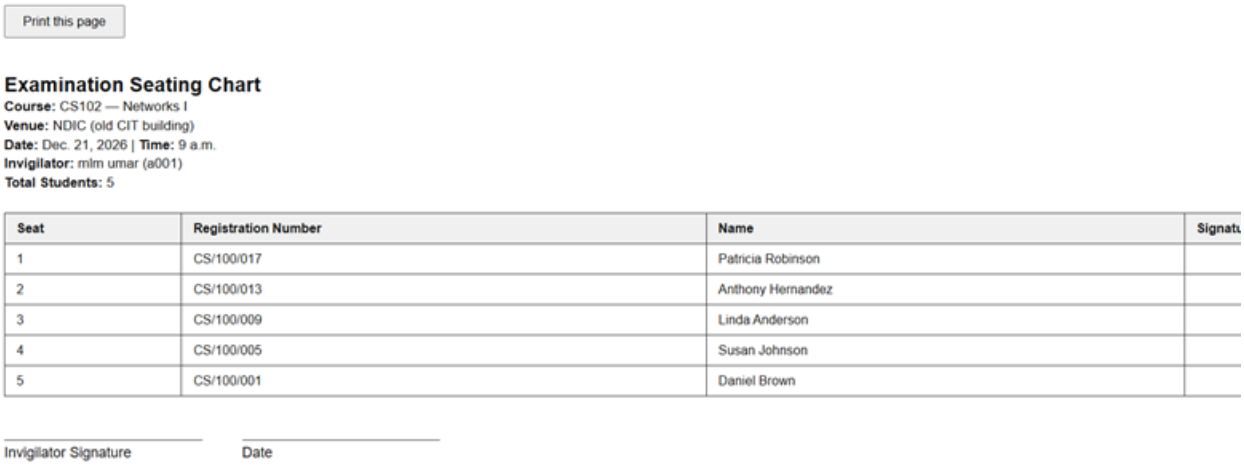


Figure 4.9: Printed seating chart

Student dashboard. A student sees the registration number, department and level, the number of courses, the exams completed and remaining, and the seat allocation. The schedule table shows each course with its date, time, venue, seat number and status. Until the seat is available, the page tells the student to check the venue one hour before the exam. The student can view the timetable, download it as a PNG or JPG file, and open the official timetables.

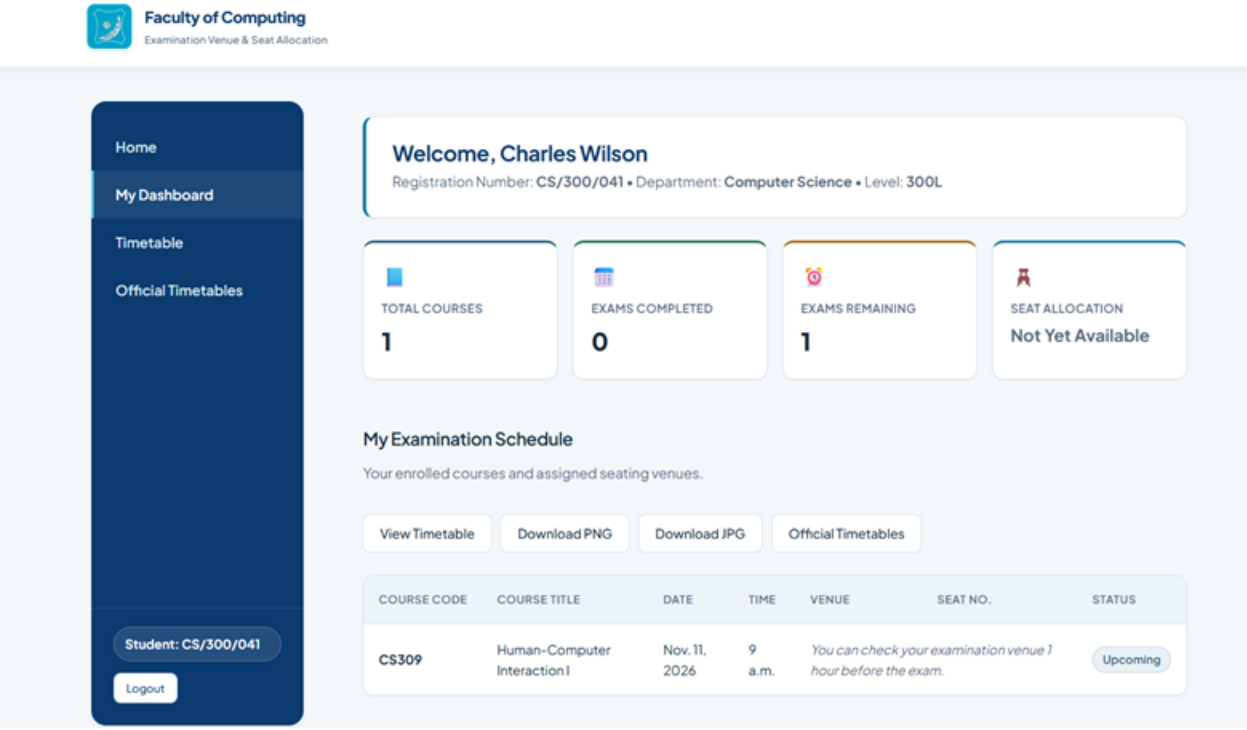


Figure 4.10: Student dashboard

Personal timetable. The student can print a personal timetable. It shows the name, registration number, department and level of the student, and a table with the course code, course title, date and time, sorted by time.



Figure 4.11: Printable personal timetable

Official timetables. This page lists the official timetables published by the exam office. The student can download each one as a PNG or JPG image, or go back to the personal timetable.



Figure 4.12: Official examination timetables

Invigilator dashboard. An invigilator sees the staff number and the number of sessions assigned, completed and remaining. The table lists each assigned session with the course, date, time, venue, duration and status, and a button to view the roster of students for that session.

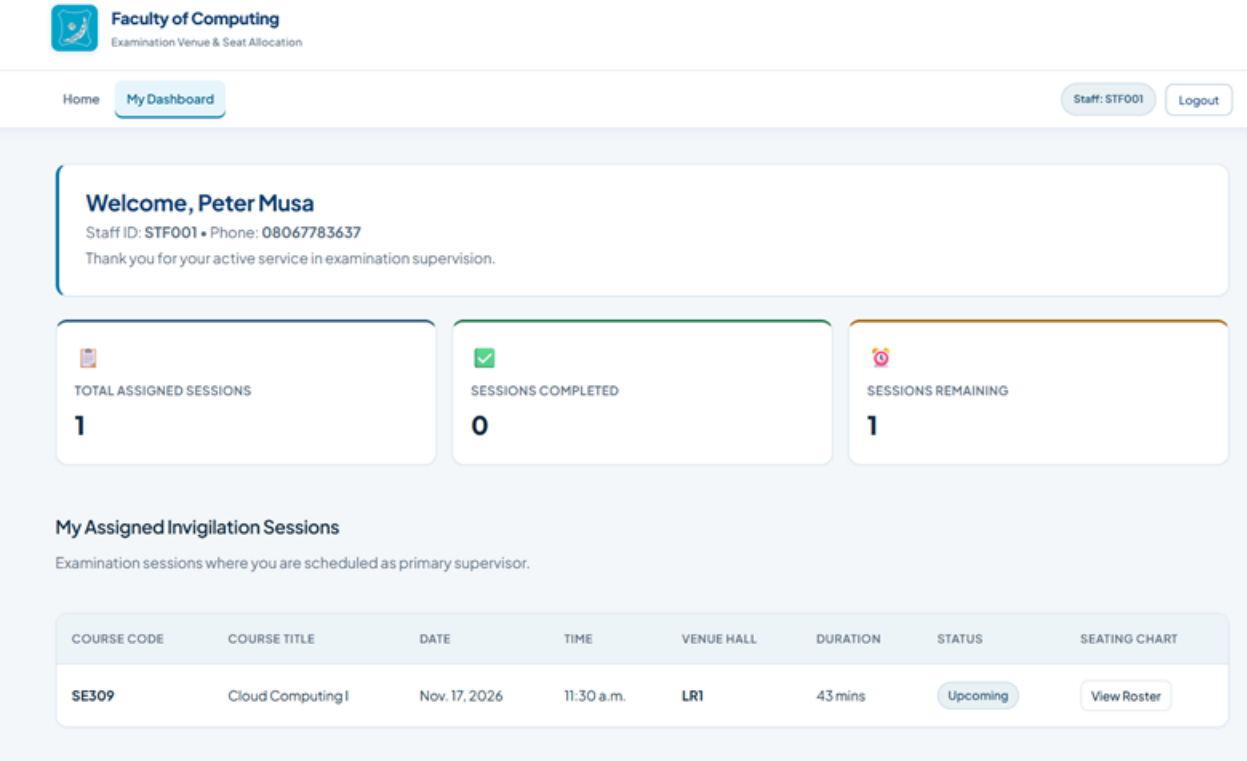


Figure 4.13: Invigilator dashboard

4.3 Testing

The system was tested to check that each part works as expected and that the parts work together. Four kinds of test were done: unit testing, integration testing, system testing and usability testing.

4.3.1 Unit Testing

Unit testing checks one small part of the system on its own, such as one function or one form. The tests were written with the Django test framework and are run with the command python manage.py test. Each test creates its own test database, so the real data is not touched. The tests cover login, students, course registration, venues, invigilators, examinations, allocation, clash detection, timetable publishing, the official timetable upload, the printable timetable and the officer accounts.

When the whole set was run, 126 tests were found, 126 ran in 287 seconds, and all of them passed. Table 4.2 shows the result.

Table 4.2: Result of the automated tests

Number of tests	Passed	Failed	Time taken
126	126	0	287 seconds

One test deliberately uploads a file that is not a real Excel file. The system rejects it and prints the message "Could not read Excel import file". This is the expected result of that test and not an error.

4.3.2 Integration Testing

Integration testing checks that the parts of the system work together. Several of the tests above follow a full path through the pages, the forms and the database. For example, they import the students, register them for a course, add an examination, allocate the seats and then check that the student sees the correct venue and seat number. They also check that a timetable with a clash, or with students who have no seat, cannot be published.

4.3.3 System Testing

System testing checks the whole system from the point of view of a user. The test cases in Table 4.3 were carried out by using the system in the browser.

Table 4.3: System test cases

No.	Test case	Expected result	Actual result	Pass or fail
1	Log in with a correct username and password	The officer goes to the officer dashboard	The officer was taken to the officer dashboard	Pass
2	Log in with a wrong password	An error message is shown and access is refused	An error message about an incorrect password was shown	Pass
3	Import students from a file	The new students are saved and logins are created	One student was imported and one login was created	Pass
4	Import a course list with a wrong registration number	The row is reported as not found and nothing wrong is saved	The registration number XX/999/999 was listed as not found in the system. The two real students were already registered, so none were added	Pass

No.	Test case	Expected result	Actual result	Pass or fail
5	Replace a course list	A preview is shown and the change is made only after confirmation. If seats are already allocated, the replace is refused	For a course without seats, a preview listed the students to add, the students to remove and the student not found. For a course with seats, the replace was refused with a message	Pass
6	Publish a timetable with two overlapping exams for the same students	Publishing is refused and the clashes are listed	The page said that 5 schedule clashes were found and listed each student's registration number, the two courses, their times and the date	Pass
7	Allocate examinations	Every registered student gets a venue and a seat number	Both examinations were allocated and each student got a venue and a seat number	Pass
8	Publish a timetable with an exam that has no venue	Publishing is refused and the problem is listed	The page said that CS101 had no venue assigned	Pass
9	Publish a timetable with students who have no seat	Publishing is refused and the problem is listed	The page said that only 7 of 8 registered students of CS309 had a seat	Pass
10	Publish a correct timetable	The status changes to Published	After the clash was removed and the exams were allocated, the page said that the timetable is published and students can see it	Pass
11	Student logs in after publishing	The student goes to the student dashboard	The student was taken to the student dashboard	Pass
12	Student downloads the official timetable	A PNG or JPG file is downloaded	The timetable file was downloaded	Pass
13	View the personal timetable	The personal timetable is shown	The personal timetable of the student was shown	Pass
14	Disable an officer account	The officer can no longer log in	The disabled officer could not log in, and a message asked for the correct username and password	Pass
15	Reset an officer's password	The officer can log in with the new password	The system said the password was reset, and the officer could log in with the new password	Pass

4.3.4 Usability Testing

Usability testing checks how easy the system is to use. A group of users tried the system and then answered a questionnaire. Each statement was marked from 1 (strongly disagree) to 5 (strongly agree). The questionnaire is in the Appendix.

The statements were:

1. The system is easy to learn.
2. The pages are easy to understand.
3. I could find the venue and seat number quickly.
4. The error messages help me to know what to do.
5. The system saves time compared with the old way.
6. I would use this system again.

Table 4.4: Usability test results (n = 25)

Statement	Average score (out of 5)
1. The system is easy to learn	4.24
2. The pages are easy to understand	4.28
3. I could find the venue and seat number quickly	4.16
4. The error messages help me to know what to do	3.56
5. The system saves time compared with the old way	4.32
6. I would use this system again	4.40

Twenty-five students took part in the usability test. After using the system, each of them rated six statements from 1 (strongly disagree) to 5 (strongly agree). The average scores ranged from 3.56 to 4.40, and the overall average was 4.16, which falls between "agree" and "strongly agree". The highest score was for using the system again (4.40). The statements on saving time (4.32), understanding the pages (4.28), learning the system (4.24) and finding the venue and seat number quickly (4.16) were also rated well. The lowest score was for the error messages (3.56), which shows that the messages need to be clearer. All the respondents were students, so the views of invigilators and the examination officer are not included.

4.4 Summary

This chapter described how the Examination Venue and Seat Allocation System was implemented and tested. It listed the tools used, gave the algorithms for allocating students, detecting clashes, publishing a timetable and importing a course list, and showed the main pages of the system. The 126 automated tests all passed. The system tests and the usability test checked the system from the point of view of its users.

CHAPTER FIVE
SUMMARY, CONCLUSION, LIMITATION AND RECOMMENDATION

5.1 Summary

The project started from a simple problem. In many universities, venues, seats and invigilators are still allocated by hand or with tools that are hard to use. This takes time and leads to mistakes, and some students cannot find where they should sit on the day of the examination. Chapter One set out this problem, together with the aim and objectives of the study.

Chapter Two reviewed ten earlier studies on examination hall allocation and seating arrangement, presented in order of year. Chapter Three described how the system was planned. The requirements were collected through an interview with the examination officer of the Faculty of Computing on 26 October 2026, and they were turned into a use case diagram, an activity diagram, a class diagram and a database design. The system was built as a Django web application with an SQLite database.

Chapter Four described how the system was built and tested. The system lets the examination officer import students and course lists from Excel files, create a timetable as a draft, add examinations to it, and allocate venues, seat numbers and invigilators. It checks for clashes before a timetable is published. Students and invigilators log in to see their own examinations. A student sees the course, date and time once the timetable is published, and the venue and seat number one hour before the examination starts. Testing was done at three levels. All 126 automated tests passed, all 15 system tests passed, and 25 students tried the system and answered a short usability questionnaire, and their average rating was 4.16 out of 5.

5.2 Conclusion

The project set out to build a system that handles venue allocation, seat numbering and invigilator assignment for the Faculty of Computing, and this has been done. The system allocates students to venues and gives each student a random seat number. It assigns an available invigilator to each venue, and it refuses to publish a timetable if a student has no seat, an examination has no venue, or two examinations clash for the same student. Students and invigilators can log in at any time to see their examinations, and the examination officer can upload the official timetable for students to download.

These features answer the needs the examination officer gave in the interview. They also address the two problems reported about the current process: students who cannot find their venue or seat, and the extra work of checking venues by hand. Every registered student is given a seat before a timetable can be published, and a student can check the allocation whenever they need to. The tests show that the system works as designed with the data used in testing. It has not yet been used for a real examination period, so its performance with the full Faculty data still has to be confirmed.

5.3 Limitation

The system works for the purpose it was built for, but it has some limits. They are stated here so that the results are not read as stronger than they are.

• It does not create timetables. The Faculty prepares the examination timetable, and the examination officer uploads it. The system does not choose dates or times.

• Passwords are weak and fixed for students and invigilators. A student's username and starting password are both the registration number, and an invigilator's are both the staff ID. Only examination officers and administrators can change their passwords. Someone who knows a registration number could log in as that student and see that student's examinations.

• The release time cannot be changed on screen. The one hour before the examination is set in one place in the code. The examination officer cannot change it from the system.

• Invigilator assignment is simple. The system looks only at who is free and who has done the fewest duties. It does not consider an invigilator's preferred times, and it does not place the course lecturer among the invigilators for that course. The examination officer mentioned both of these as wishes.

• The results depend on the lists that are uploaded. If a student is missing from a course list, for example a carryover or spillover student, that student will not get a seat. When a student import contains rows with no registration number, those rows are skipped without a clear warning.

• Testing was done on a small scale. The tests used test data, not the full Faculty of about 2,000 students, and the usability questionnaire was answered by 25 students only, so the views of invigilators and the examination officer are not included. The system has not been tested with many users at the same time.

5.4 Recommendation

The following steps are suggested for anyone who continues this work, starting with the ones that matter most.

1. Improve password security. Ask students and invigilators to change their password the first time they log in, and let them reset it if they forget it.

2. Try the system in a real examination period. Use the full Faculty data and many users at once, and move to a stronger database than SQLite if the load requires it.

3. Add a settings page for the release time. The examination officer should be able to change the one hour before the examination from the system, without changing the code.

4. Make invigilator assignment fairer. Allow invigilators to give the times that suit them, and add the rule that the course lecturer is one of the invigilators for that course.

5. Report import problems clearly. Show the officer which rows were skipped and why, and find a way to include carryover and spillover students in course lists.

6. Send notifications. A text message or an email with the venue and seat shortly before the examination would help students who do not log in, as one of the reviewed studies did with text messages (Adetona & Akintoye, 2023).

7. Generate timetables. A later version could suggest examination dates and times, and avoid clashes before the officer uploads the timetable.

REFERENCES

Adetona, S., Hassan, E., Salawu, R., & Omolola, S. (2020). The Development of a Web-based Application of Examination Seating Arrangement for Student. *ABUAD Journal of Engineering Research and Development*. 3(1),23-33.

Adetona, S. O., & Akintoye, F. A. (2023). Design and implementation of an examination seating arrangement application to curb examination malpractice. ABUAD Journal of Engineering Research and Development, 6(2), 143–156. https://doi.org/10.53982/ajerd.2023.0602.14-j

Anjum, S., Chodey, M. D., & Afzal, M. C. (2021). Automation of exam hall allotment and seating arrangement. International Journal of Engineering Research & Technology, 10(6), 447–452. https://www.ijert.org/research/automation-of-exam-hall-allotment-and-seating-arrangement-IJERTV10IS060193.pdf

Dener, M., & Calp, M. H. (2018). Solving the exam scheduling problems in central exams with genetic algorithms. Mugla Journal of Science and Technology, 4(1), 102–115. https://doi.org/10.22531/muglajsci.423185

Inamdar, A., Gangar, A., Gupta, A., & Shrivastava, V. (2018). Automatic exam seating & teacher duty allocation system. In 2018 Second International Conference on Inventive Communication and Computational Technologies (ICICCT) (pp. 1302–1306). IEEE.

Kashyap, M. M., Thejas, S., Gaurav, C. G., & Srinivas, K. S. (2021). Exam seating allocation to prevent malpractice using genetic multi-optimization algorithm. Communications in Computer and Information Science, 1366, 131–145. https://doi.org/10.1007/978-981-16-0419-5_11

Kibona, I. E., January, J., Matimbwa, H., Nchia, E., Matungwa, W., & Vuai, S. A. H. (2026). Examination seating optimization using graph coloring and combinatorial design. Array, 30, Article 100897. https://doi.org/10.1016/j.array.2026.100897

Mora, H., Sulaiman, U. I., & Musa, F. N. (2023). The design and implementation of online examination hall allocation system for students in Kaduna Polytechnic, *Kaduna. International Journal of Scientific Advances, 4(4*), 553–562. https://doi.org/10.51542/ijscia.v4i4.9

Papaioannou, E., Vardakis, A., & Kaklamanis, C. (2018). oPESA: Online platform for automatic exam-hall seat allocation. International Journal of Engineering Technologies and Management Research, 5(6), 51–65. https://doi.org/10.29121/ijetmr.v5.i6.2018.245

Shelar, P. S. (2018). Exam Cell Automation System. *International Journal for Research in Applied Science and Engineering Technology*, *6*(5), 1186–1188. https://doi.org/10.22214/ijraset.2018.5193

Subhashini, S., Sreerambabu, J., & Riyaz, M. M. (2022). Online Examination Hall Seating Arrangement. *International Journal for Research in Applied Science and Engineering Technology*, *10*(8), 962–964. https://doi.org/10.22214/ijraset.2022.46312

Tung, S. N., Jaafar, J. B., Aziz, I. A., Nguyen, H. G., & Bui, A. N. (2021). Genetic algorithm for solving multi-objective optimization in examination timetabling problem. International Journal of Emerging Technologies in Learning, 16(11), 4–24. https://doi.org/10.3991/ijet.v16i11.21017

Vakil, M., Deepak, Chirag, Kumar, H., & Tyagi, D. (2025). Exam hall seating arrangement system using MERN stack. International Journal of Innovative Research in Technology, 11(12), 2121–2124.

Appendix A: Interview Questions Used to Gather Requirements (Section 3.4.2)

A.1 Examination Officer interview

These are the 24 questions asked in the interview with the Examination Officer of the Faculty of Computing, Bayero University Kano, on 26 October 2026.

Section A: Background and current process

#	Question
1	What is your role in the Faculty, and how long have you held it?
2	How many students, courses and departments does the Faculty have each semester?
3	How do you get the list of students registered for each course?
4	How is the examination timetable prepared, and who approves it?
5	How do you decide which venue each course uses? Do you check venue capacity?
6	How are students given seat numbers? Is the seating random, in order, or by some rule?
7	How are invigilators chosen and assigned to venues and examinations?
8	How do students find out their venue and seat?

Section B: Problems with the current process

#	Question
9	Which tools do you use now: paper, Excel, WhatsApp or any software?
10	What goes wrong most often when venues and seats are allocated?
11	Have two examinations ever been scheduled at the same time for the same students? What happened?
12	Have venues ever been overcrowded or left half empty? How did you handle it?
13	How much time does the allocation take each semester, and how many people work on it?
14	Have students ever complained about not finding their seat or venue?

Section C: What the new system must do

#	Question
15	What would you most want an examination allocation system to do for you?
16	Who should be able to use the system, and what should each person see?
17	Should the system assign invigilators automatically, or should you assign them yourself?
18	What information should the student see about their examination, and when?
19	Should students see the venue and seat only shortly before the examination, or earlier?
20	What reports do you need, for example a seating list for each venue or an attendance sheet?
21	In what form do you hold student and course lists today, and would you be able to supply them as Excel files?
22	Do you want to upload the official timetable as a picture or PDF so students can download it?

Section D: Closing

#	Question
23	Is there anything else the system should do, or any rule it must follow?
24	May I come back to you if I need to confirm any of your answers?

A.2 Student interview

These are the questions sent to students of the Faculty of Computing to find out the problems they faced with the existing way of finding their examination venue and seat. Five students replied.

#	Question
1	How do you find out your exam venue and seat now?
2	What problems did you face with the current way of finding your venue and seat?
3	Have you ever failed to find your seat or venue? What happened?
4	Did you have to look for the exam officer to find out your seat? How long did it take?
5	After you checked your venue once, could you check it again later? What happened?
6	Has anyone you know ever had a clash, or two exams at the same time?
7	What would you like a new system to show you, and when?

Appendix B: Usability Questionnaire and Results (Section 4.3.4)

The questionnaire below was given to the users after they tried the system (Section 4.3.4). It was answered by 25 users. The average score for each statement is in Table 4.4, and the overall average rating was 4.16 out of 5.

Usability Questionnaire: Examination Venue and Seat Allocation System, Faculty of Computing, Bayero University Kano

This questionnaire is part of a final year project. Please use the system and then answer the questions. Your answers will be used for the project only. You do not need to write your name.

Part A: About you

1. I am a: [ ] Student [ ] Exam Officer [ ] Invigilator [ ] Lecturer [ ] Other: _____

2. I used the system as: [ ] Student [ ] Officer [ ] Both

Part B: Your opinion

Tick one box for each statement. 1 = strongly disagree, 2 = disagree, 3 = not sure, 4 = agree, 5 = strongly agree.

Statement	1	2	3	4	5
1. The system is easy to learn.	[ ]	[ ]	[ ]	[ ]	[ ]
2. The pages are easy to understand.	[ ]	[ ]	[ ]	[ ]	[ ]
3. I could find the venue and seat number quickly.	[ ]	[ ]	[ ]	[ ]	[ ]
4. The error messages help me to know what to do.	[ ]	[ ]	[ ]	[ ]	[ ]
5. The system saves time compared with the old way.	[ ]	[ ]	[ ]	[ ]	[ ]
6. I would use this system again.	[ ]	[ ]	[ ]	[ ]	[ ]

Part C: Your comments

7. What did you like most about the system?

8. What should be improved?

Appendix C: Code of the Major Functions (Section 4.2.2)

Each part below matches one algorithm in Section 4.2.2. The code is taken from the system. Part C.4 is shortened where marked.

C.1 Algorithm 1: Allocate students for one examination (allocations/services.py)

```
def _order_venues_for_allocation(venues, needed):
    """
    Venues that can seat the whole group come first: the smallest hall that
    still fits, so a five-student exam doesn't take a 120-seat theatre while
    small rooms sit empty. Least-used breaks ties between halls of the same
    size. Venues too small for the group follow, largest first, so the
    overflow needs as few extra rooms as possible.
    """
    usage_counts = {v.id: ExamSession.objects.filter(venue=v).count() for v in venues}
    fitting = sorted(
        (v for v in venues if v.capacity >= needed),
        key=lambda v: (v.capacity, usage_counts[v.id]),
    )
    overflowing = sorted(
        (v for v in venues if v.capacity < needed),
        key=lambda v: (-v.capacity, usage_counts[v.id]),
    )
    return fitting + overflowing



def _order_invigilators_by_load(invigilators):
    """
    Same idea as _order_venues_by_load, but for invigilators: whoever
    has done the fewest duties so far gets offered the next one first.
    """
    usage_counts = {inv.id: ExamSession.objects.filter(invigilator=inv).count() for inv in
invigilators}
    return sorted(invigilators, key=lambda inv: usage_counts[inv.id])



@transaction.atomic
def allocate_exam(exam, venues=None, invigilators=None):
    """
    Full allocation for a single exam. Raises ValueError if there isn't
    enough free venue capacity, or not enough free invigilators to staff
    the venues that get used.
    """
    students = get_registered_students(exam)
    if not students:
        raise ValueError(f"No students are registered for {exam.course.course_code}.")

    random.shuffle(students)

    # Idempotent: wipe this exam's previous sessions (Allocation rows cascade).
    ExamSession.objects.filter(exam=exam).delete()

    candidate_venues = _order_venues_for_allocation(
        list(venues or Venue.objects.all()), len(students)
    )
    free_venues = [v for v in candidate_venues if not venue_is_busy(v, exam)]

    candidate_invigilators = _order_invigilators_by_load(list(invigilators or
Invigilator.objects.all()))

    remaining = students[:]
```

```
    sessions_created = []
    used_invigilator_ids = set()

    for venue in free_venues:
        if not remaining:
            break

        free_invigilator = next(
            (inv for inv in candidate_invigilators
             if inv.id not in used_invigilator_ids and not invigilator_is_busy(inv, exam)),
            None,
        )
        if free_invigilator is None:
            continue  # no one free for this venue right now — try the next venue

        session = ExamSession.objects.create(exam=exam, venue=venue,
invigilator=free_invigilator)
        sessions_created.append(session)
        used_invigilator_ids.add(free_invigilator.id)

        batch, remaining = remaining[:venue.capacity], remaining[venue.capacity:]
        Allocation.objects.bulk_create([
            Allocation(student=student, exam_session=session, seating_number=i + 1)
            for i, student in enumerate(batch)
        ])

    if remaining:
        raise ValueError(
            f"Not enough free venue capacity/invigilators for "
            f"{exam.course.course_code}: {len(remaining)} student(s) "
            f"could not be seated. Add more venues or free up invigilators."
        )

    return sessions_created
```

C.2 Algorithm 2: Detect clashes between examinations (exams/clash_service.py)

```
def _clashes_among(exams):
    """
    Every (student, overlapping exam pair) clash among `exams`.

    Two queries — the exam list and one registration query — then all
    comparisons happen in memory, so the cost does not grow with
    exams x registrations x courses.
    """
    exams = list(exams)
    if len(exams) < 2:
        return []

    exams_by_course = defaultdict(list)
    for exam in exams:
        exams_by_course[exam.course_id].append(exam)

    student_courses = defaultdict(set)
    students = {}
    for registration in Registration.objects.filter(
        course_id__in=exams_by_course
    ).select_related("student"):
        student_courses[registration.student_id].add(registration.course_id)
        students[registration.student_id] = registration.student

    # Students who take the same courses share the same exam pairs, so each
    # pair of courses is compared once for the whole group.
    students_by_courses = defaultdict(list)
    for student_id, course_ids in student_courses.items():
```

```
        if len(course_ids) > 1:
            students_by_courses[tuple(sorted(course_ids))].append(student_id)

    clashes = []
    for course_ids, student_ids in students_by_courses.items():
        for course_a, course_b in combinations(course_ids, 2):
            for exam_a in exams_by_course[course_a]:
                for exam_b in exams_by_course[course_b]:
                    if not exams_overlap(exam_a, exam_b):
                        continue
                    if exam_a.exam_time <= exam_b.exam_time:
                        first, second = exam_a, exam_b
                    else:
                        first, second = exam_b, exam_a
                    for student_id in student_ids:
                        clashes.append({
                            "student": students[student_id],
                            "exam_a": first,
                            "exam_b": second,
                        })
    return clashes



def find_timetable_clashes(timetable):
    """
    Clashes that publishing this batch would expose to students: pairs where
    at least one exam belongs to the batch. Clashes between two published
    batches elsewhere are not this batch's problem.
    """
    batch_exam_ids = set(timetable.exams.values_list("id", flat=True))
    if not batch_exam_ids:
        return []

    scope = Exam.objects.filter(
        Q(timetable=timetable) | Q(timetable__status="PUBLISHED")
    ).select_related("course")

    return [
        clash
        for clash in _clashes_among(scope)
        if clash["exam_a"].id in batch_exam_ids
        or clash["exam_b"].id in batch_exam_ids
    ]
```

C.3 Algorithm 3: Publish a timetable (exams/views.py)

```
@staff_required
def publish_timetable(request, timetable_id):
    """
    Publishing makes the batch visible to students, so it refuses an empty
    batch, an exam whose registered students are not seated, and any pair
    of exams whose times overlap for a student.
    """
    timetable = get_object_or_404(Timetable, id=timetable_id)

    if request.method != "POST":
        return redirect("timetable_detail", timetable_id=timetable.id)

    if timetable.status != "DRAFT":
        messages.error(
            request,
            f"Only a draft timetable can be published. '{timetable.name}' is "
            f"{timetable.get_status_display().lower()}.",
        )
        return redirect("timetable_detail", timetable_id=timetable.id)
```

```
    exams = list(timetable.exams.select_related("course"))
    if not exams:
        messages.error(
            request,
            f"'{timetable.name}' has no exams yet, so there is nothing to "
            "publish. Upload its exams first.",
        )
        return redirect("timetable_detail", timetable_id=timetable.id)

    seating_problems = _seating_problems(timetable, exams)
    if seating_problems:
        messages.error(
            request,
            f"Cannot publish — {len(seating_problems)} exam(s) cannot seat "
            "their registered students. Allocate seating, then publish again:",
        )
        for exam, reason in seating_problems:
            messages.warning(request, f"{exam.course.course_code}: {reason}")
        return redirect("timetable_detail", timetable_id=timetable.id)

    clashes = find_timetable_clashes(timetable)
    if clashes:
        messages.error(
            request,
            f"Cannot publish — {len(clashes)} schedule clash(es) found. "
            "Fix these before publishing:",
        )
        for clash in clashes:
            messages.warning(
                request,
                f"{clash['student'].registration_number}: "
                f"{clash['exam_a'].course.course_code} "
                f"({clash['exam_a'].exam_time:%H:%M}) overlaps "
                f"{clash['exam_b'].course.course_code} "
                f"({clash['exam_b'].exam_time:%H:%M}) on "
                f"{clash['exam_a'].exam_date}.",
            )
        return redirect("timetable_detail", timetable_id=timetable.id)

    timetable.status = "PUBLISHED"
    timetable.save(update_fields=["status"])
    messages.success(request, f"'{timetable.name}' is now published — students can see it.")
    return redirect("timetable_detail", timetable_id=timetable.id)
```

C.4 Algorithm 4: Import the students registered for a course (students/views.py)

```
def import_registrations(request):
    """
    Upload the list of students registered for ONE course.

    The officer picks a course from a dropdown and uploads a CSV or Excel
    file with the columns reg_no and full_name (one student per row). Every
    student found in the system is registered for that course.

    Replace mode never deletes straight from the upload. It first shows a
    preview of exactly who will be added and removed; the change is only
    applied after the officer confirms. A file that matches nobody is
    rejected outright, so a wrong reg format can't empty a course.
    """
    # ... (form and file reading left out) ...
            elif data is not None and replace and Allocation.objects.filter(
                exam_session__exam__course=course
            ).exists():
                # Safety rule: if seats were already given out for this course,
                # removing students from its list would leave seats belonging
                # to students who are no longer registered. So we refuse.
                messages.error(
```

```
                    request,
                    "Seats have already been allocated for this course, so its list can't be "
                    "replaced. Upload without 'Replace' to add students instead.",
                )
            # ... (lines left out) ...
            elif data is not None:
                # Build a lookup of every student by registration number,
                # in lowercase, so CST/22/SWE/00858 matches cst/22/swe/00858.
                # One database query here is much faster than one per row.
                students_by_reg = {
                    s.registration_number.strip().lower(): s for s in Student.objects.all()
                }

                # The students already registered for THIS course
                existing_ids = set(
                    Registration.objects.filter(course=course).values_list("student_id",
flat=True)
                )

                to_add = []          # Student objects to register
                listed_ids = set()   # students seen in the file (to catch repeats)
                already = bad_rows = 0
                unknown = []         # (reg_no, name) pairs not found in the system
                name_mismatch = []   # students found, but the name in the file looks different

                for row in data.rows:
                    # Read the reg number from the row. The extra names let the
                    # file use other common column headings too.
                    reg_no = _get_field(
                        row, "reg_no", "registration_number", "registration number",
                        "matric_number", "matric number",
                    )
                    # Read the student's full name (may be empty if the column is missing)
                    full_name = _get_field(
                        row, "full_name", "full name", "name", "student_name", "student name",
                    )

                    # Blank row: nothing to read, count it and move on
                    if not reg_no:
                        bad_rows += 1
                        continue

                    # Find the student. None means no such student exists yet.
                    student = students_by_reg.get(reg_no.lower())
                    if student is None:
                        unknown.append((reg_no, full_name))
                        continue

                    # The same student listed twice in one file: ignore the repeat
                    if student.id in listed_ids:
                        continue
                    listed_ids.add(student.id)

                    # If the file gave a name, check it against the system's name.
                    # A mismatch usually means a typo in the reg number.
                    if full_name and not _names_match(full_name, student):
                        name_mismatch.append((reg_no, full_name, f"{student.first_name}
{student.last_name}"))

                    if student.id in existing_ids:
                        # Already registered for this course: nothing to do
                        already += 1
                    else:
                        # New: keep it for the preview or the append below
                        to_add.append(student)
```

```
added = len(to_add)
remove_ids = sorted(existing_ids - listed_ids)
# ... (replace-mode messages left out) ...
elif replace:
    # Hold the exact add/remove sets until the officer confirms.
    request.session[PENDING_REPLACE_SESSION_KEY] = {
        "course_id": course.id,
        "add_ids": [student.id for student in to_add],
        "remove_ids": remove_ids,
        "filename": request.FILES["csv_file"].name,
        "created_at": timezone.now().isoformat(),
    }
    # ... (preview details left out) ...
else:
    # Append mode only ever adds rows, so it is safe to save now.
    with transaction.atomic():
        Registration.objects.bulk_create(
            [Registration(student=student, course=course) for student in to_add],
            ignore_conflicts=True,
        )
```